



Republic of Tunisia
Ministry of Higher Education and Scientific Research
University of Tunis El Manar
National Engineering School of Tunis



ELECTRICAL ENGINEERING DEPARTMENT

End of year project II

Design and implementation of a flight controller on SoC for a UAV

Developed by
Ahmed KALLEL
Nerimen SAHLI

Group
2AGE1

Defended on May 2, 2026

Before the Jury :

President	:	Mr. Mohamed Mehdi FILALI
Examiner	:	Mr. Lotfi CHARAABI
Supervisor	:	Mr. Joseph HAGGEGE
Co-Supervisor	:	Mrs. Faten BEN ABDALLAH

Academic year : 2025–2026

Acknowledgments

We would like to express our deepest gratitude to our supervisors, **Mr. Joseph Haggege** and **Mrs. Faten Ben Abdallah**, for their invaluable guidance, patience, and mentorship throughout this project. Their technical expertise and constant encouragement were essential to the successful completion of this work.

We extend our sincere thanks to the members of the jury **Mr. Mohamed Mehdi Filali** and **Mr. Lotfi Charaabi** for their time and effort in reviewing this work. We truly appreciate their presence and look forward to their insightful feedback and contributions.

We would also like to thank the **National Engineering School of Tunis (ENIT)**, and specifically the **Department of Electrical Engineering** and the **Research Laboratory in Automatic Control (LARA)**, for providing the environment and resources necessary to bring this project to life.

Finally, we wish to thank our families and friends for their unconditional love, sacrifices, and continuous support. Their belief in us has been our greatest source of motivation throughout this entire journey.

Abstract

This project focuses on the design and implementation of a comprehensive flight controller for a quadrotor drone, utilizing a Zynq-7010 System-on-Chip (SoC) architecture. The study begins with rigorous mechanical modeling and the derivation of non-linear equations of motion, followed by linearization for PID control law synthesis. A hardware/software co-design methodology is adopted, partitioning critical tasks into programmable logic (FPGA) to ensure deterministic PWM signal generation and sensor interfacing, while the processing system (ARM) executes navigation algorithms and Kalman filtering. System validation is achieved through Hardware-in-the-Loop (HIL) simulation in MATLAB/Simulink prior to deployment on the physical platform. Experimental results confirm attitude stability and the controller's robustness against disturbances. This work demonstrates the effectiveness of the SoC approach in meeting the real-time requirements of aeronautical embedded systems.

Keywords : Quadrotor Drone, Flight Controller, Zynq SoC, PID Control, Kalman Filter.

Contents

General Introduction	8
Introduction	8
1 Theoretical Foundations and System Modeling	9
1.1 Introduction	9
1.2 Overview and Analysis of Existing Solutions	9
1.3 Proposed Solution Specifications	12
1.3.1 Functional Specifications	12
1.3.2 Technical and Hardware Partitioning	12
1.3.3 Development and Validation Tools	13
1.4 Mechanical Preliminary	13
1.4.1 Force and Moment Translation to the Center of Gravity	14
1.5 Physical Configuration and Modeling	15
1.5.1 Quadrotor Configuration and Reference Frames	18
1.5.2 Mechanical and Aerodynamic Modeling	19
1.5.3 Linearization of Equations of Motion	20
1.5.4 State-Space Representation	22
1.6 Embedded System Architecture (Xilinx Zynq)	24
1.6.1 Hardware/Software Partitioning Strategy	28
1.6.2 Peripheral Configuration: PWM, I ² C and UART	30
1.7 Conclusion	36
2 Implementation and Experimental Results	38
2.1 Introduction	38
2.2 Actuator Control and Telemetry	38
2.2.1 PWM Signal Generation (PL Implementation)	38
2.2.2 Propulsion System, ESC Interfacing, and Power Distribution	40
2.2.3 Wireless Control	43
2.3 System Identification	44
2.3.1 Determination of Moments of Inertia	44
2.4 Sensor Interfacing and Calibration	46
2.4.1 Sensor Technology and Physical Principles	46
2.4.2 Sensor Calibration	47
2.4.3 Data Acquisition Implementation	50
2.5 Data Fusion and Control Design	51
2.5.1 Kalman Filter Implementation for State Estimation	51
2.5.2 PID Controller Tuning and Coefficient Determination	52
2.6 Testing and Performance Analysis	54
2.6.1 Hardware-in-the-Loop (HIL) Simulation and Validation	54

2.6.2	Closing the Loop: Flight Results	57
2.7	Conclusion	60
Conclusion and Future Work		61
Bibliography		62

List of Figures

1.1	The trend of the number of publications about multicopters [19]	11
1.2	A quadcopter in hovering flight.	14
1.3	The process of translating the thrust and moments of Propeller #1 to the center of the quadcopter: the reaction torque translates directly as a free vector, while the thrust force generates residual rolling and pitching moments at the CoG [19].	15
1.4	Quadcopter's configuration and geometry parameters	16
1.5	Forces on quadcopters	17
1.6	Block diagram - Flight Control System approximated to the quadcopter	22
1.7	Zybo Board	25
1.8	Zynq 7000 SoC Functional Blocks	26
1.9	Power Supply Simplified Schematic	27
1.10	Complete Vivado IP Integrator block design for the Zynq-7010 SoC. This overview highlights the AXI Interconnect architecture bridging the Processing System (PS) with the Programmable Logic (PL) peripherals, including PWM generators for motor control and I2C/UART interfaces.	29
1.11	Architectural comparison of PWM generation pathways. The AXI Timer provides a direct, 32-bit, jitter-free route to the physical SelectIO pins, avoiding the 16-bit TTC limitations and EMIO routing bottlenecks.	30
1.12	Zynq-7000 signal routing pathways demonstrating the physical limitations of the Multiplexed I/O (MIO) forcing secondary TTC channels through the Extended MIO (EMIO).	31
1.13	Architectural comparison of I2C bus instantiation. The PL-based AXI IIC mitigates the 16-byte FIFO interrupt bottleneck, allows for unlimited bus instantiation to avoid address conflicts, and provides precise MMCM-backed clocking.	32
1.14	Data reception architecture for the ESP-01S Wi-Fi module. Utilizing a PL-based AXI UART dedicated strictly to reception allows for a deep, BRAM-backed FIFO that buffers large network packets, significantly reducing CPU interrupt overhead compared to the hard-wired 64-byte PS UART limit.	33
1.15	Post-implementation resource utilization summary on the Zynq-7010, showing the lightweight footprint of the PL-based peripheral architecture.	34
1.16	Vivado device floorplan showing the physical placement and routing of the AXI-based peripherals within the Programmable Logic, with cyan cells indicating implemented logic resources.	35
1.17	On-chip power estimation report from Vivado, illustrating the dominant contribution of the Processing System (PS7) and the negligible overhead of the hardware-offloaded peripherals.	36
2.1	Oscilloscope observations of PWM Signals at different duty cycles	39
2.2	Signal transmission of the propulsor model [19]	41
2.3	Propulsion System Components	41

2.4	Propulsion System Wiring	42
2.5	ESP-01S Module	43
2.6	Experimental setup for the torsional pendulum method using the quad-rotor.	45
2.7	Comparative setup for torsion constant calibration using a reference cylinder.	45
2.8	MPU6050 Sensor	46
2.9	HMC5883L Sensor	46
2.10	I2C Master-Slave Pass-Through architecture	50
2.11	Response of a typical PID closed loop system to a step function as set point[17]	53
2.12	Parameterization of the 6-DOF block highlighting the experimentally derived mass and inertia matrix.	54
2.13	Comprehensive Simulink HIL Block Diagram. The architecture includes the UART serial communication interface for Zybo integration, the 6-DOF rigid body dynamics plant model, and the real-time synchronization kernel.	55
2.14	3D visualization of the simulated quadcopter orientation in the NED coordinate frame.	56
2.15	Complete State Machine for UAV Flight Controller	58

List of Tables

1.1	Summary of existing small aircraft solutions and autopilot platforms [19][18][3] . . .	11
1.2	Zybo Board Components	25
1.3	Zybo onboard power supply rails.	27
2.1	AT commands utilized for ESP-01S communication and configuration [6].	44

General Introduction

Unmanned aerial vehicles have transitioned from niche research platforms into ubiquitous tools that are reshaping industries ranging from infrastructure inspection to precision agriculture and emergency response. Among the many configurations available, the quad-rotor has emerged as the dominant architecture in both the consumer and research markets, owing to its mechanical simplicity, vertical take-off and landing capability, and the decoupled controllability of its four independent rotors. Yet beneath this apparent simplicity lies a tightly coupled system of aerodynamics, embedded electronics, and real-time control theory that demands rigorous engineering at every level of the design.

This project aims to design, implement, and validate a custom-built quad-rotor flight controller from first principles. In the perspective of reverse-engineering an off-the-shelf autopilot solution, the work deliberately spans the full development chain — from the equations of motion to the physical flight tests — with the aim of building a thorough and practical understanding of each subsystem and the interactions between them.

The report is organised into two chapters. The first establishes the theoretical and architectural foundations of the system: the quad-rotor's nonlinear equations of motion are derived from Newton's laws, linearised about the hover trim condition using small perturbation theory, and cast into a state-space representation suitable for controller synthesis. In parallel, the Xilinx Zynq-7010 System-on-Chip is presented as the target hardware platform, and a principled hardware/software partitioning strategy is defined to meet the deterministic timing requirements of a flight controller.

The second chapter addresses the practical realisation of this design. Proceeding from the hardware interfaces upward, it covers PWM signal generation, ESC and propulsion system integration, wireless command reception via the ESP-01S Wi-Fi module, experimental identification of the quad-rotor's moments of inertia, sensor calibration, and the implementation of a discrete-time Kalman filter for attitude estimation. A Hardware-in-the-Loop simulation environment developed in MATLAB/Simulink is then used to validate and tune the PID control laws before their deployment on the physical platform, with the chapter concluding with an analysis of the flight test results.

Taken together, the two chapters form a study of the development cycle of an embedded flight controller, from mathematical model to flying hardware.

Chapter 1

Theoretical Foundations and System Modeling

1.1 Introduction

In the evolving world of unmanned aerial vehicles (UAVs), quad-rotor drones stand out as being one of, if not the most recognized and most commonly used category of drones. Their versatility and adaptability across different industries has made them indispensable in this new technological era.

Quad-rotors have earned this reputation thanks to their vertical takeoff and hovering capabilities, eliminating the need for runways or launchers, as well as their superior manoeuvrability making them convenient for close-range inspections and obstacle dense areas.

They are adaptable to specific mission requirement and can be upgraded to offer stable aerial imaging, multiple sensor integrations as well as intelligent route planning and automated tasks.

Beyond their four rotor arms and flying capabilities lies a sophisticated mechanism built on aerodynamics, embedded systems, and control theory. These sophisticated systems are evaluated according to several criteria, including payload capacity, flight time and endurance, environmental tolerance like wind resistance and integration with software.

Quad-rotors are redefining how data is captured, decisions are made, and complex challenges dealt with. In the near future, the emerging trends focus on AI integration, in order to detect objects or take autonomous decisions mid-flight for example.

The primary objective of this chapter is to define the theoretical premise as well as the embedded foundation, based on which the actual system will be built on.

1.2 Overview and Analysis of Existing Solutions

The development of small unmanned aircraft systems has progressed through several distinct phases, giving rise to three primary categories of aerial platforms: fixed-wing aircraft, single-rotor helicopters, and multicopters [19]. Each configuration presents a unique set of trade-offs across the five core performance dimensions — ease-of-use, reliability, maintainability, time of endurance, and payload capacity — and understanding these trade-offs is essential to contextualising the design choices made in contemporary solutions [19].

Comparative Analysis of Existing Platforms

Fixed-wing aircraft offer superior endurance and reasonable payload capacity, but their inability to hover and their demand for large operational airspaces make them poorly suited for applications requiring precision station-keeping or confined-area operation [19]. Single-rotor helicopters achieve greater payload capacity than multicopters, but their strongly coupled flight dynamics and highly nonlinear behaviour introduce significant complexity in both manual piloting and autopilot design, reducing their overall reliability and raising maintenance costs [18][3].

Multicopters, by contrast, have emerged as the dominant solution in the consumer and prosumer markets due to their decoupled flight axes, mechanical simplicity, and VTOL capability [19]. Multicopters outperform the other two configurations in ease-of-use, reliability, and maintainability, while lagging in endurance and payload capacity [19]. These latter limitations, however, have proven addressable through engineering compromises: endurance can be extended through battery replacement, and payload capacity can be increased by scaling propeller count or diameter [19].

Commercial and Open-Source Solutions

The commercial landscape has evolved from early high-cost professional systems such as the Microdrones MD4-200 (2006) and Draganflyer X6 (2008), which were largely inaccessible to mass consumers, toward integrated ready-to-fly platforms [19]. The Parrot AR.Drone (2010) marked a pivotal transition by incorporating onboard optical flow sensing, ultrasonic altitude measurement, and smartphone-based control, substantially lowering the barrier to entry [19]. The DJI Phantom (2012) further matured this trajectory by adding GPS-based position hold, wind resistance, and camera payload capability, catalysing widespread adoption in aerial photography [19].

On the research and development side, a rich ecosystem of open-source autopilot frameworks — including Ardupilot, Pixhawk, OpenPilot, and MikroKopter — emerged during the 2010–2013 period [2], democratising access to advanced flight control capabilities and accelerating academic and hobbyist innovation alike.

Identified Bottlenecks

Despite their advantages, existing multicopter solutions face inherent physical limitations when scaled upward [19]. Larger propeller radii result in slower dynamic rotor response and increased mechanical fatigue at the rotor hub due to blade-flapping effects — phenomena that undermine the very simplicity that makes multicopters attractive [19]. While solutions such as the Volocopter VC200 demonstrate that manned-scale multicopters are technically feasible through propeller multiplication, these approaches compromise reliability, endurance, and maintainability, and introduce significant airworthiness certification burdens [19]. The consensus in the field therefore favours small to micro-scale multicopters, where the structural and control simplicity of the platform can be fully exploited without the drawbacks associated with scaling [3].

This analysis establishes that while existing solutions have reached a high level of maturity in the small-scale consumer segment, meaningful design opportunities remain in further improving energy efficiency, payload-to-weight ratio, and autonomous capability [18] — areas that continue to drive active research and product development (As seen in figure 1.1).

Summary of Existing Solutions

Table 1.1 provides a consolidated overview of the platforms and solutions discussed in this section, highlighting their respective strengths and limitations [19][18].

Table 1.1: Summary of existing small aircraft solutions and autopilot platforms [19][18][3]

Solution		Type	Period	Strengths	Limitations
Fixed-Wing Aircraft		Platform	—	Best endurance, good payload	No hover, large space needed
Single-Rotor Helicopter		Platform	—	High payload capacity	Nonlinear dynamics, high maintenance
Multicopter (general)		Platform	—	Ease-of-use, reliability, maintainability	Limited endurance and payload
Keyence Rosaucer	Gy-	Consumer product	Early 1990s	First mini quadcopter	~3 min flight, very limited
Roswell Flyer / Draganflyer		Consumer product	1999–2008	Early autopilot integration	Expensive, no GPS
Microdrones MD4-200		Commercial product	2006	Reliable commercial benchmark	High cost, niche market
Parrot AR.Drone		Consumer product	2010	Easy to pilot, SDK, indoor hover	No GPS, limited range
DJI Phantom		Consumer product	2012	GPS hold, aerial photography	Moderate endurance
Volocopter VC200		Scaled multicopter	~2015	Manned multicopter proof-of-concept	Heavy, airworthiness burden
Ardupilot / Pixhawk		Open-source autopilot	2010–2013	Lowered development threshold	Requires integration expertise

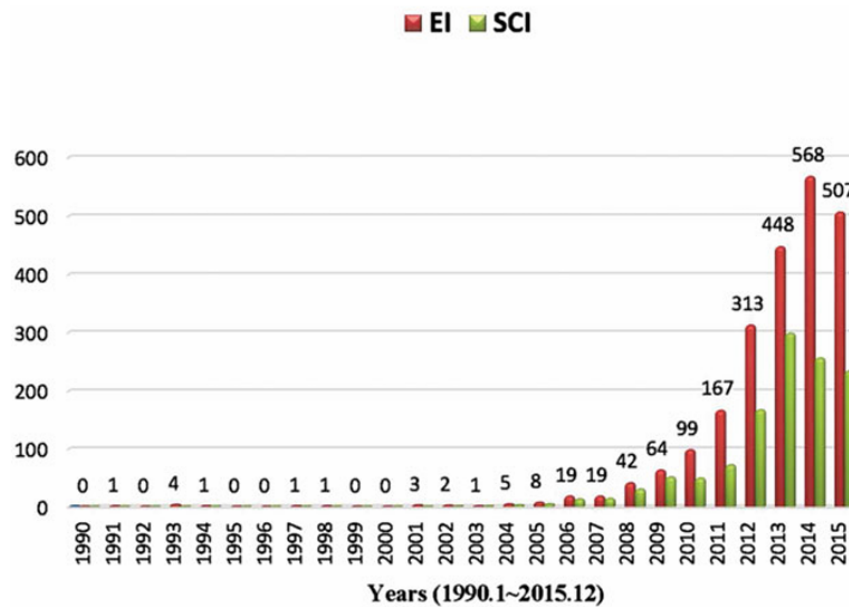


Figure 1.1: The trend of the number of publications about multicopters [19]

1.3 Proposed Solution Specifications

The objective of this project is to develop a robust, high-performance flight controller by leveraging a Hardware/Software (HW/SW) co-design methodology. By utilizing the Zynq-7010 System-on-Chip (SoC), the solution partitions tasks between the Processing System (PS) and Programmable Logic (PL) to meet the deterministic demands of flight stability.

1.3.1 Functional Specifications

The final product, which is the flight controller, should be meeting the following functional requirements:

- **Real-Time Attitude Estimation:** Implementation of a sensor fusion algorithm (Kalman Filter) to process raw data received from the MPU6050+HMC5883L IMU, providing accurate Euler angles (Roll, Pitch, Yaw).
- **Deterministic Control Loops:** an appropriate integration of nested PID control loops for attitude and altitude stabilization, ensuring a smooth response to the commands of the user.
- **Wifi Communication:** Integration of an ESP-01S transceiver for low-latency, long-range command reception from a ground station.
- **PWM Actuation:** High-resolution Pulse Width Modulation (PWM) generation to drive the four brushless motors via Electronic Speed Controllers (ESCs).

1.3.2 Technical and Hardware Partitioning

The system architecture follows a strategic HW/SW partitioning to optimize performance:

Processing System (PS) - Software Layer

The ARM Cortex-A9 processor is responsible for the "intelligence" of the drone:

- **Noise filtering:** converting the raw data received from the sensors to noiseless and accurate Euler's angles to determine an accurate position of the current state.
- **Navigation Algorithms:** Calculation of the error between the pilot's set-points and the previously stated current state.
- **Control Logic:** Execution of the discrete-time PID control laws.

Programmable Logic (PL) - Hardware Layer

The FPGA fabric handles time-critical tasks that require absolute determinism:

- **Hardware PWM IP Cores:** Dedicated modules to ensure stable signal timing for the motors, independent of CPU load.
- **Sensor Interfacing:** Hardware-level I2C management to reduce CPU overhead during data acquisition.
- **AXI Interconnect:** High-speed data exchange between the software and hardware layers using AXI4-Lite and AXI-Stream protocols.
- **communication with the transceiver via UART:** Hardware-level UART management to reduce CPU overhead in terms of command reception from the ESP to the embedded environment.

1.3.3 Development and Validation Tools

The implementation is supported by a professional toolchain to ensure reliability:

- **Vivado Design Suite:** For the creation of the IP Integrator block design and hardware synthesis.
- **Vivado SDK:** For developing the embedded C/C++ application running on the ARM processor.
- **MATLAB/Simulink:** Used for the Hardware-in-the-Loop (HIL) simulation to validate the control gains before deployment.

Designing a flight controller requires a comprehensive understanding of the quadrotor's movement in three-dimensional space. By linking its mechanical dynamics to a state-space representation, we bridge the gap between the theoretical aspect and the practical implementation of control laws on a System-on-Chip.

1.4 Mechanical Preliminary

At first glance, a quadcopter — or multicopter — derives its mechanical behavior directly from its physical structure. It is commonly configured in either an **X** or **+** arrangement, with four propellers mounted on motorized rotors positioned at the extremities of the airframe.

Thrust is generated by spinning these propellers at high rotational speeds, driven by the electromagnetic torque that each motor's stator exerts on its corresponding rotor. Since the stators are rigidly attached to the airframe, understanding how their reaction torques interact with the overall structure is fundamental to grasping the drone's flight mechanics.

Assuming the airframe behaves as a perfectly rigid body, Newton's Third Law dictates that for every torque applied to a rotor, an equal and opposite reaction torque is transmitted back into the airframe. Because a moment is a *free vector* — meaning it can be translated to any point along its axis without introducing residual forces or moments — all four reaction torques effectively act about the drone's central vertical axis. Consequently, if all four motors were to spin in the same direction, their reaction torques would cumulatively sum, driving the airframe into an uncontrolled rotation about the yaw axis, a condition that would render the vehicle entirely uncontrollable [18][19].

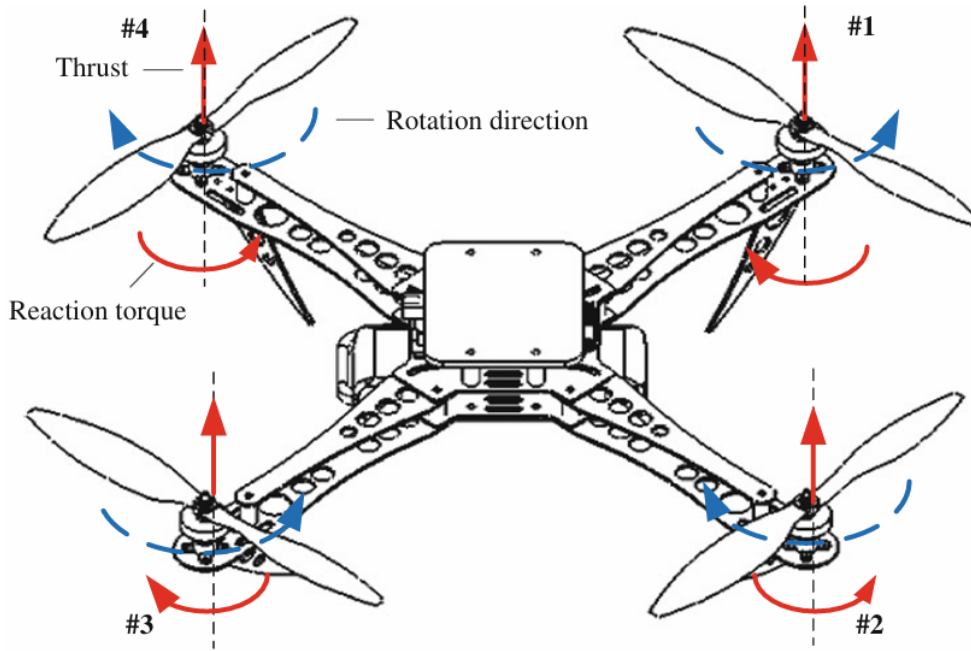


Figure 1.2: A quadcopter in hovering flight.

To counteract this, the rotors are arranged in two counter-rotating pairs, as illustrated in Figure 1.2:

- **Propellers #1 and #3** rotate **counter-clockwise**
- **Propellers #2 and #4** rotate **clockwise**

In this configuration, the reaction torques of each pair cancel one another out, yielding a net yaw moment of zero and establishing the foundational condition for stable, controlled hovering.

1.4.1 Force and Moment Translation to the Center of Gravity

An additional consideration of fundamental importance concerns the point of application of the thrust forces. Although each propeller generates its thrust at its own location on the airframe, all forces and moments must be expressed at the center of gravity (CoG) of the structure for the equations of motion to be properly formulated.

Within the framework of *screw theory* (or *torseur* formalism), the mechanical state of Propeller #1 at its hub P is fully described by the following screw, comprising a thrust force along the vertical axis and a reaction torque about the same axis:

$$\mathcal{T}_P = \begin{Bmatrix} \mathbf{F}_P \\ \mathbf{M}_P \end{Bmatrix} = \begin{Bmatrix} F \mathbf{e}_z \\ \tau_r \mathbf{e}_z \end{Bmatrix} \quad (1.1)$$

Translating this screw to the CoG introduces a residual moment, defined as the cross product of the position vector $\mathbf{r} = r_x \mathbf{e}_x + r_y \mathbf{e}_y$ from the CoG to the propeller hub, and the thrust force:

$$\mathbf{r} \times \mathbf{F}_P = (r_x \mathbf{e}_x + r_y \mathbf{e}_y) \times (F \mathbf{e}_z) = F r_y \mathbf{e}_x - F r_x \mathbf{e}_y \quad (1.2)$$

The resulting residual moment carries two orthogonal horizontal components: a component along $\mathbf{e}_x$ and a component along $\mathbf{e}_y$, which correspond respectively to a **rolling moment** and a **pitching moment** at the CoG. The translated screw therefore takes the form:

$$\mathcal{T}_{\text{CoG}} = \begin{Bmatrix} \mathbf{F}_P \\ \mathbf{M}_{\text{CoG}} \end{Bmatrix} = \begin{Bmatrix} F e_z \\ \underbrace{F r_y}_{\text{rolling}} \mathbf{e}_x - \underbrace{F r_x}_{\text{pitching}} \mathbf{e}_y + \underbrace{\tau_r}_{\text{yaw}} \mathbf{e}_z \end{Bmatrix} \quad (1.3)$$

The reaction torque τ_r , being a free vector, translates unmodified to the central axis and contributes solely to the yaw moment. This process is illustrated in Figure 1.3 for Propeller #1.

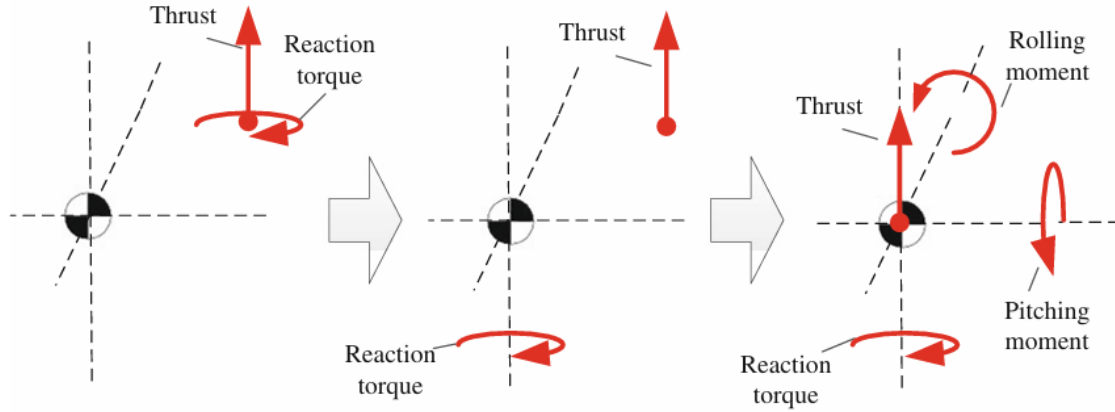


Figure 1.3: The process of translating the thrust and moments of Propeller #1 to the center of the quadcopter: the reaction torque translates directly as a free vector, while the thrust force generates residual rolling and pitching moments at the CoG [19].

This effect applies equally to all four propellers. However, by virtue of the symmetric arrangement of the rotors about the CoG, the induced pitching and rolling moments of opposing propellers are equal and opposite, and cancel one another when the thrust allocation is properly regulated — provided a rigorous and unified mechanical formalism governs all four rotors simultaneously.

1.5 Physical Configuration and Modeling

The physical layout of the quadrotor is not merely a structural choice — it directly conditions the validity of the mathematical model and the performance of the control laws derived from it. Several geometric and mechanical considerations must therefore be established before the equations of motion are formulated.

Airframe Geometry and Arm Length

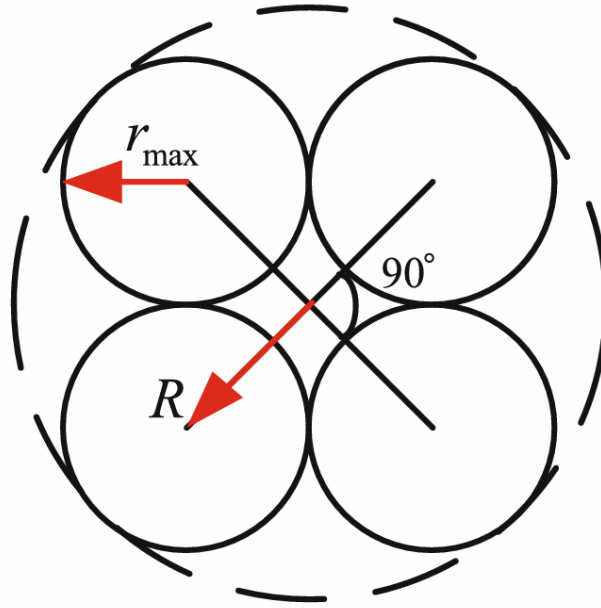


Figure 1.4: Quadcopter's configuration and geometry parameters

The first consideration concerns the relationship between the propeller dimensions and the airframe geometry. As illustrated in Figure 1.4, the size of the airframe is closely tied to the radius of the propellers. For a standard quadcopter configuration, the four arms are separated by an angle $\theta = 90$, and the geometric relationship between the airframe radius R (equivalent to the arm length L used in the equations of motion) and the effective propeller disc radius $r_{\max}$ is given by:

$$R = \frac{r_{\max}}{\sin \frac{\theta}{2}} = \frac{r_{\max}}{\sin \frac{180}{n}} \quad (1.4)$$

where n is the number of arms. For a quadcopter, $n = 4$ and $\theta = 90$, which simplifies to $R = r_{\max}$ [19].

However, the effective disc radius $r_{\max}$ is not identical to the physical propeller radius r_p . Aerodynamic interference between adjacent propellers reduces thrust efficiency when the propeller discs overlap or are placed too close to one another. Experimental investigations have shown that when the spacing between propellers ranges from $0.1 r_{\max}$ to $r_{\max}$, each propeller maintains an independent vortex without significantly degrading performance. To balance compactness against aerodynamic efficiency, the following design rule is adopted:

$$r_{\max} = 1.05 r_p \sim 1.2 r_p \quad (1.5)$$

This establishes a direct link between the propeller sizing — which is itself determined by the required thrust and payload — and the arm length L that appears explicitly in the moment equations of the dynamic model (see Equation (1.15)).

Motor Orientation and Thrust Direction

A second structural decision concerns the orientation of the motors relative to the airframe. Two conventions exist in practice:

- **Motors facing downward:** the propellers operate in a pushing configuration, generating thrust directed upward along the positive Z_B body axis. This arrangement produces a clean, unobstructed downwash and offers the additional practical advantages of protecting the motors from rain and ensuring accurate barometric altitude readings, since the downwash remains below the sensor plane.
- **Motors facing upward:** the propellers operate in a pulling configuration. While this orientation offers improved propeller protection during landing and a wider camera field of view, the thrust vector remains aligned with $+Z_B$, leaving the equations of motion formally unchanged.

In both cases, the thrust produced by each rotor is aligned with the Z_B axis of the body frame, which is consistent with the force and moment model adopted in Equation (1.16).

Center of Gravity Height and Aerodynamic Stability

The vertical position of the center of gravity (CoG) along the Z_B axis has a significant influence on the aerodynamic behavior of the quadrotor during forward flight and under wind disturbance. As illustrated in Figure 1.5, the drag forces acting on the airframe generate a moment about the CoG whose sign depends on the relative position of the CoG with respect to the aerodynamic pressure center:

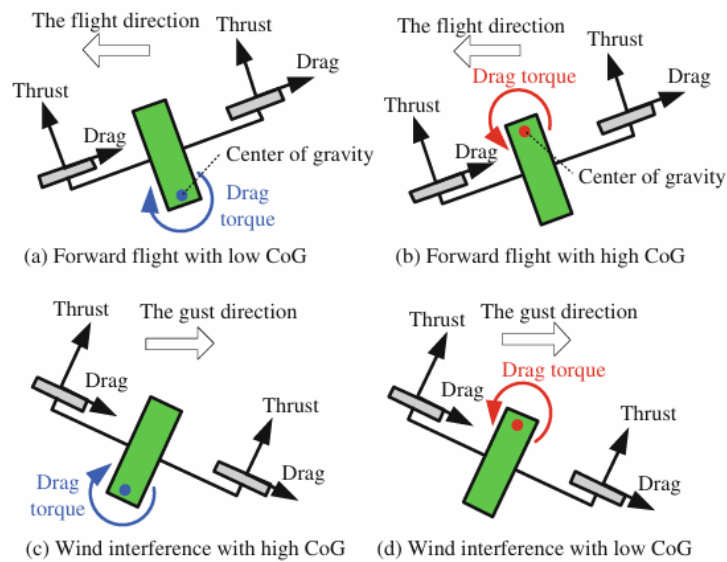


Figure 1.5: Forces on quadcopters

- A **low CoG** places the center of gravity below the point of application of the resultant drag force, producing a self-correcting, stabilizing pitching moment that tends to restore the quadrotor to its nominal attitude.
- A **high CoG** inverts this relationship, yielding a destabilizing moment that amplifies any attitude perturbation, demanding greater corrective authority from the controller.

This consideration justifies the modeling assumption that the CoG coincides with the geometric center of the symmetric airframe, which simultaneously minimizes the destabilizing moment arm and simplifies the inertia tensor to a diagonal matrix $I = \text{diag}(I_{xx}, I_{yy}, I_{zz})$, as used in Equation (1.8).

Autopilot Placement and the Lever-Arm Effect

A final but critical consideration concerns the physical placement of the autopilot unit, which integrates the Inertial Measurement Unit (IMU) — the accelerometers and gyroscopes responsible for state estimation. The ideal installation position is exactly at the geometric center of the airframe, coinciding with the CoG.

When the IMU is offset from the CoG by a position vector $\mathbf{d} = d_x \mathbf{e}_x + d_y \mathbf{e}_y + d_z \mathbf{e}_z$, the measured acceleration $\mathbf{a}_{\text{meas}}$ differs from the true CoG acceleration $\mathbf{a}_{\text{CoG}}$ by spurious centrifugal and tangential components:

$$\mathbf{a}_{\text{meas}} = \mathbf{a}_{\text{CoG}} + \dot{\boldsymbol{\omega}} \times \mathbf{d} + \boldsymbol{\omega} \times (\boldsymbol{\omega} \times \mathbf{d}) \quad (1.6)$$

where $\boldsymbol{\omega}$ is the angular velocity vector of the body frame and $\dot{\boldsymbol{\omega}}$ its time derivative. This phenomenon, known as the *lever-arm effect*, introduces a systematic bias into the state estimation that degrades the accuracy of the feedback signals on which the controller relies. For this reason, the autopilot must be mounted as close as possible to the CoG, with its principal axis aligned with the X_B forward direction of the body frame. In the context of this model, the lever-arm offset $\mathbf{d}$ is assumed to be zero, which is consistent with the rigid-body, CoG-centered formulation of the equations of motion presented in the following sections.

1.5.1 Quadrotor Configuration and Reference Frames

The first step in modeling the quadrotor is to identify the variables that define its kinematics and orientation.

By varying the motor's speed and direction of rotation, the quadrotor can change its position and orientation. There are 12 states that describe the quadrotor's dynamic behaviour, divided into four groups of three variables:

- space position: $X = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$
- linear velocity: $V = \begin{pmatrix} u \\ v \\ w \end{pmatrix}$
- rotational angles: $\Omega = \begin{pmatrix} \phi \\ \theta \\ \psi \end{pmatrix}$
- angular velocities: $\boldsymbol{\omega} = \begin{pmatrix} p \\ q \\ r \end{pmatrix} = \begin{pmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{pmatrix}$

These states can be considered as the quadrotor's outputs. The inputs of the quadrotor are the applied forces F and torques M generated from the four motors.

The dynamic model of the quadrotor is derived by considering it a rigid body in a 3-dimensional space, with a symmetrical quadrotor frame and a center of gravity located at the geometric center. The propellers are assumed to be rigid due to their relatively small diameter (25.5 cm).

The wind and ground effect disturbances are neglected, so the quadrotor will fly in a calm environment and at a proper altitude.

Finally, the gyroscopic effect is neglected, because the mass of the propellers (9 g each) is negligible compared to the total quadrotor mass of 1250 g.

For the quadrotor design, we define the reference systems as:

- Earth reference system (Earth frame): X_E, Y_E, Z_E
- Quadrotor reference system (Body frame): X_B, Y_B, Z_B

The transformations between the two systems are defined by three Euler angles:

- ϕ (Roll) = angle of rotation along axis X_B to X_E
- θ (Pitch) = angle of rotation along axis Y_B to Y_E
- ψ (Yaw) = angle of rotation along axis Z_B to Z_E

The derivatives of ϕ , θ , and ψ with respect to time are the angular rotation speeds of the system $\dot{\phi}$ (Roll rate), $\dot{\theta}$ (Pitch rate), and $\dot{\psi}$ (Yaw rate), also noted as p , q , and r respectively. [18]

1.5.2 Mechanical and Aerodynamic Modeling

The quadrotor body axis equations of motion can be derived from Newton's second law of motion as shown in Equations (1.7) and (1.8):

$$F = \begin{pmatrix} F_x \\ F_y \\ F_z \end{pmatrix} = ma = m(\dot{V} + \omega \times V) = m \left[\begin{pmatrix} \dot{u} \\ \dot{v} \\ \dot{w} \end{pmatrix} + \begin{pmatrix} p \\ q \\ r \end{pmatrix} \times \begin{pmatrix} u \\ v \\ w \end{pmatrix} \right] = m \begin{pmatrix} \dot{u} + qw - rv \\ \dot{v} + ru - pw \\ \dot{w} + pv - qu \end{pmatrix} \quad (1.7)$$

where m is the quadrotor mass.

$$M = \begin{pmatrix} M_x \\ M_y \\ M_z \end{pmatrix} = I\dot{\omega} + \omega \times (I\omega) = \begin{pmatrix} I_{xx}\dot{p} - (I_{yy} - I_{zz})qr \\ I_{yy}\dot{q} - (I_{zz} - I_{xx})pr \\ I_{zz}\dot{r} - (I_{xx} - I_{yy})pq \end{pmatrix} \quad (1.8)$$

where $I = \begin{pmatrix} I_{xx} & 0 & 0 \\ 0 & I_{yy} & 0 \\ 0 & 0 & I_{zz} \end{pmatrix}$ is the inertia matrix and I_{xx} , I_{yy} , I_{zz} are the rotational resistances of roll, pitch, and yaw respectively.

The lift force is distributed between the four rotors, and transmitted to the propellers. The relationship between the forces (F_1, F_2, F_3, F_4) and the rotation speed of the propellers ($\omega_1, \omega_2, \omega_3, \omega_4$) is given by Equation (1.9):

$$F_i = k\omega_i^2 \quad (1.9)$$

where $i \in \{1, 2, 3, 4\}$ and k is a proportionality coefficient.

The moments (M_1, M_2, M_3, M_4) are calculated using Equation (1.10):

$$M_i = L \times F_i \quad (1.10)$$

where $i \in \{1, 2, 3, 4\}$ and L is the arm distance from the rotor axis to the quadrotor's center of gravity.

The amount of force and torque produced from each propeller can be described by Equations (1.11) and (1.12), where F_{th} is the rotor thrust and T is the torque:

$$F_{th} = C_{F_{th}} \rho \pi R^4 \Omega^2 \quad (1.11)$$

$$T = C_T \rho \pi R^5 \Omega^2 \quad (1.12)$$

C_T and $C_{F_{th}}$ are the rotor thrust and torque coefficients, ρ is air density, R is the propeller radius, and Ω is the angular rate of the rotor.

We can simplify the equations by considering $k_{F_{th}}$ and k_T as the resulting thrust and torque coefficients, resulting in Equations (1.13) and (1.14):

$$F_{th} = k_{F_{th}} \Omega^2 \quad (1.13)$$

$$T = k_T \Omega^2 \quad (1.14)$$

To implement these equations in an input matrix, we define δ_{thr} as the thrust input, δ_{ail} as the lateral input, δ_{ele} as the longitudinal input, and δ_{rud} as the directional input. The quadrotor motors are numbered from 1 to 4, where Ω_i is the angular rate of rotor i .

We obtain the input matrix shown in Equation (1.15):

$$\begin{pmatrix} \delta_{thr} \\ \delta_{ail} \\ \delta_{ele} \\ \delta_{rud} \end{pmatrix} = \begin{pmatrix} k_{F_{th}}(\Omega_1^2 + \Omega_2^2 + \Omega_3^2 + \Omega_4^2) \\ k_{F_{th}}L(-\Omega_2^2 + \Omega_4^2) \\ k_{F_{th}}L(\Omega_1^2 - \Omega_3^2) \\ k_T(-\Omega_1^2 + \Omega_2^2 - \Omega_3^2 + \Omega_4^2) \end{pmatrix} \quad (1.15)$$

The quadrotor dynamics can then be described by the system of Equations (1.16), which incorporate the gravitational force in the quadrotor body-axis equations of motion:

$$\begin{aligned} \dot{u} &= rv - qw - g \sin \theta + a_x \\ \dot{v} &= pw - ru + g \sin \phi \cos \theta + a_y \\ \dot{w} &= qu - pv + g \cos \phi \cos \theta + \frac{\delta_{thr}}{m} \\ \dot{p} &= \frac{(I_{yy} - I_{zz})qr + \delta_{ail}}{I_{xx}} \\ \dot{q} &= \frac{(I_{zz} - I_{xx})pr + \delta_{ele}}{I_{yy}} \\ \dot{r} &= \frac{(I_{xx} - I_{yy})pq + \delta_{rud}}{I_{zz}} \end{aligned} \quad (1.16)$$

These equations take into consideration the projection of the Earth's gravity vector onto the quadrotor's body axis using Euler angles θ and ϕ , along with aerodynamic accelerations a_x and a_y , the thrust acceleration $\left(\frac{\delta_{thr}}{m}\right)$ aligned with the Z axis, and the inputs δ_{ail} , δ_{ele} , and δ_{rud} generating the moments M_x , M_y , and M_z .

While these non-linear equations accurately describe the quadrotor's physics, they must be linearized to create a state-space model.[18]

1.5.3 Linearization of Equations of Motion

In order to linearize the quadrotor's physics equations for controller design, we consider the drone at a hovering position. When hovering, the roll and pitch angles are zero, the angular velocities are zero, and the thrust perfectly balances gravity. We can therefore apply *small perturbation theory*, an analytical technique that linearizes complex non-linear systems by assuming disturbances from a steady state are small, allowing higher-order terms to be neglected. By applying this theory, all coupled non-linear terms become infinitely small and can be cancelled out, simplifying the equations.[18]

The stable hover state defines four trim conditions:

- Linear and angular velocities are zero: $u_0 = v_0 = w_0 = 0$ and $p_0 = q_0 = r_0 = 0$.
- Roll and pitch angles are zero: $\phi_0 = \theta_0 = 0$. The yaw angle ψ_0 is set to zero for simplicity.
- The total thrust balances the gravitational pull: $\delta_{thr,0} = -mg$.

We assume that each state variable undergoes a minor deviation (Δ) from its hover trim condition. For example, the velocity u has a deviation Δu , meaning:

$$u = u_0 + \Delta u = \Delta u$$

The trigonometric functions can be approximated using the first-order Taylor series:

$$\sin(\Delta\theta) \approx \Delta\theta, \quad \cos(\Delta\theta) \approx 1, \quad \sin(\Delta\phi) \approx \Delta\phi, \quad \cos(\Delta\phi) \approx 1.$$

While aerodynamic accelerations (a_x, a_y) and moments are not entirely neglected, they are linearized using aerodynamic stability derivatives.

Applying these approximations to the non-linear equations (1.16) results in the linearized equations of motion:

$$\begin{cases} m\Delta\dot{u} = \Delta F_x - mg\Delta\theta \\ m\Delta\dot{v} = \Delta F_y + mg\Delta\phi \\ m\Delta\dot{w} = \Delta F_z \\ I_{xx}\Delta\dot{p} = \Delta M_x \\ I_{yy}\Delta\dot{q} = \Delta M_y \\ I_{zz}\Delta\dot{r} = \Delta M_z \\ \Delta p = \Delta\dot{\phi} \\ \Delta q = \Delta\dot{\theta} \\ \Delta r = \Delta\dot{\psi} \end{cases} \quad (1.17)$$

In order to define how the surrounding air resists the quadrotor's motion, we introduce stability derivatives. They quantify the extent to which an aerodynamic force or moment changes in response to small changes in velocity or angular rate. For example, F_{x_u} describes the drag relative to forward speed, while M_{x_p} represents the aerodynamic resistance to rotation:

$$F_{x_u} = \frac{1}{m} \frac{\partial F_x}{\partial u}, \quad M_{x_p} = \frac{1}{I_{xx}} \frac{\partial M_x}{\partial p} \quad (1.18)$$

We can then obtain the final linearized model:

$$\begin{cases} \Delta F_x = F_{x_u}u + F_{x_q}q + F_{x_{\delta_{ele}}} \delta_{ele} \\ \Delta F_y = F_{y_v}v + F_{y_p}p + F_{y_{\delta_{ail}}} \delta_{ail} \\ \Delta F_z = F_{z_w}w + F_{z_{\delta_{thr}}} \delta_{thr} \\ \Delta M_x = M_{x_v}v + M_{x_p}p + M_{x_{\delta_{ail}}} \delta_{ail} \\ \Delta M_y = M_{y_u}u + M_{y_q}q + M_{y_{\delta_{ele}}} \delta_{ele} \\ \Delta M_z = M_{z_r}r + M_{z_{\delta_{rud}}} \delta_{rud} \\ \Delta p = \Delta\dot{\phi} \\ \Delta q = \Delta\dot{\theta} \\ \Delta r = \Delta\dot{\psi} \end{cases} \quad (1.19)$$

A linear state-space representation of the quadrotor dynamics in hover flight can then be derived.[18]

1.5.4 State-Space Representation

In order to determine a state-space representation, the first step is to combine the equations obtained into the standard state equation $\dot{\mathbf{X}} = \mathbf{A}\mathbf{X} + \mathbf{B}\mathbf{U}$.

$\mathbf{X} = [u, v, w, p, q, r, \phi, \theta, \psi]^T$ is the state vector, including linear velocities, Euler angles, and angular velocities.

$\mathbf{U} = [\delta_{thr}, \delta_{ail}, \delta_{ele}, \delta_{rud}]^T$ is the input vector, comprising the total thrust and the three moments that affect the quadrotor, which are functions of the four motor speeds.

The **matrix A** describes the internal dynamics of the system and how the quadrotor's current state influences its future state, regardless of external input. The **matrix B** defines how the control inputs affect the derivatives of the states.

The state equation can be written as follows [18]:

$$\begin{bmatrix} \dot{u} \\ \dot{v} \\ \dot{w} \\ \dot{p} \\ \dot{q} \\ \dot{r} \\ \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} F_{x_u} & 0 & 0 & 0 & F_{x_q} & 0 & 0 & -g & 0 \\ 0 & F_{y_v} & 0 & F_{y_p} & 0 & 0 & g & 0 & 0 \\ 0 & 0 & F_{z_w} & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & M_{x_v} & 0 & M_{x_p} & 0 & 0 & 0 & 0 & 0 \\ M_{y_u} & 0 & 0 & 0 & M_{y_q} & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & M_{z_r} & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} u \\ v \\ w \\ p \\ q \\ r \\ \phi \\ \theta \\ \psi \end{bmatrix} + \begin{bmatrix} 0 & 0 & F_{x\delta_{ele}} & 0 \\ 0 & F_{y\delta_{ail}} & 0 & 0 \\ F_{z\delta_{thr}} & 0 & 0 & 0 \\ 0 & M_{x\delta_{ail}} & 0 & 0 \\ 0 & 0 & M_{y\delta_{ele}} & 0 \\ 0 & 0 & 0 & M_{z\delta_{rud}} \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \delta_{thr} \\ \delta_{ail} \\ \delta_{ele} \\ \delta_{rud} \end{bmatrix} \quad (1.20)$$

If the mathematical elaboration made in this section can be resumed in a block diagram, according to McRuer et al (1973) [15], it would be the one below (Figure 1.6):

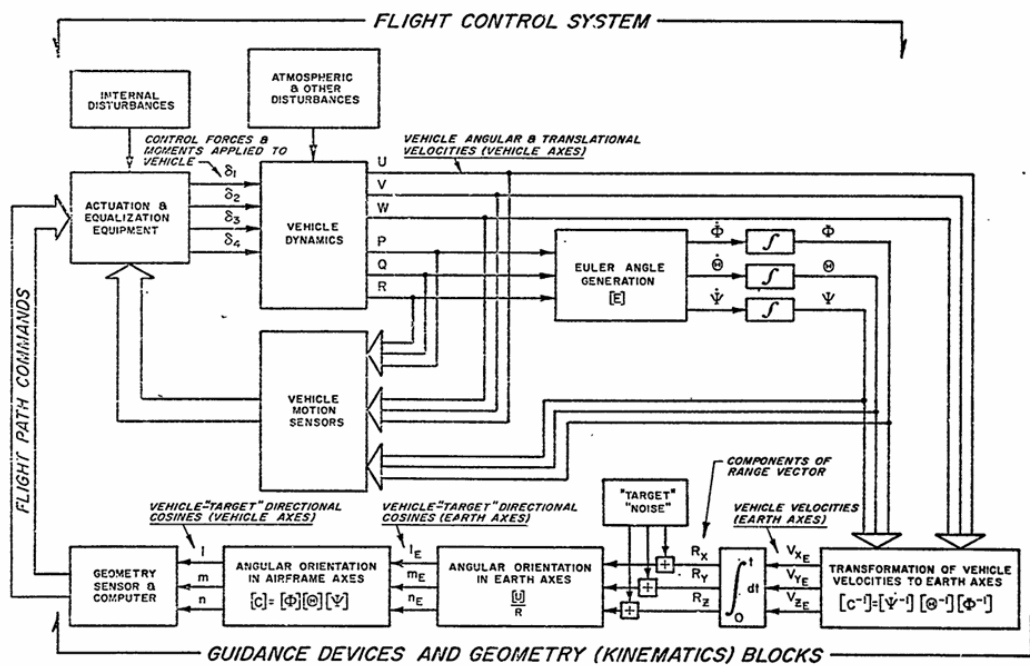


Figure 1.6: Block diagram - Flight Control System approximated to the quadcopter

This linear model will serve as the foundation for the development of the flight controller that will be implemented on the System-on-Chip architecture.

1.6 Embedded System Architecture (Xilinx Zynq)

The Zybo (Zynq Board) (Figure 1.7, Table 1.2) is an entry-level embedded development platform designed by Digilent, built around the Xilinx Zynq XC7Z010-1CLG400C System-on-Chip (SoC). Its architecture makes it a particularly well-suited candidate for applications requiring concurrent real-time processing and hardware-level signal handling, such as the flight controller developed in this work.

The Z-7010 SoC integrates a dual-core ARM Cortex-A9 processor running at 650 MHz with reconfigurable FPGA logic equivalent to a Xilinx Artix-7 device. This tight hardware coupling between a general-purpose processor and programmable logic on a single chip is a defining advantage of the Zynq platform. It allows time-critical tasks — such as the reception of wireless control data from the ESP-01S module and PWM motor output — to be offloaded entirely to dedicated hardware peripherals implemented in the FPGA fabric, while higher-level control logic such as the PID controller and flight state machine executes on the ARM cores. This separation of concerns between hardware and software significantly reduces latency and CPU overhead compared to a purely software-based microcontroller approach. The processing system includes a DDR3 memory controller interfaced with 512 MB of RAM operating at 1050 Mbps bandwidth and eight DMA channels, providing sufficient memory throughput for real-time sensor fusion and control loop execution.

A comprehensive set of peripheral controllers is available natively, encompassing Gigabit Ethernet, USB 2.0, SDIO, SPI, UART, CAN, and I2C interfaces. For this project, the I2C bus was used to communicate with the MPU-92.65 inertial measurement unit, and the PS UART interface was used for debug terminal output, hardware-in-the-loop testing, and other host-facing communication — all supported natively without any additional hardware.

The programmable logic portion of the Z-7010 provides 4,400 logic slices — each containing four 6-input look-up tables and eight flip-flops — 240 KB of distributed block RAM, 80 DSP slices, and two clock management tiles equipped with phase-locked loop (PLL) and mixed-mode clock manager (MMCM) resources. Internal clock speeds can exceed 450 MHz, enabling the AXI peripherals instantiated in the FPGA fabric to operate with the timing precision required for both PWM motor signal generation and reliable UART frame reception from the ESP-01S Wi-Fi module. Specifically, an AXI UART block was instantiated in the programmable logic to handle the ESP-01S serial data stream, isolating this communication channel from the processor system's UART and ensuring that control data acquisition does not interfere with debug or host-facing traffic.

An on-chip analog-to-digital converter (XADC) is also available should analog sensing be required in future iterations of the design. The board exposes a range of I/O interfaces including a dual-role HDMI port, a 16-bit VGA output, a tri-mode Gigabit Ethernet PHY, a MicroSD slot with Linux filesystem support, an OTG USB 2.0 PHY, an audio codec, and 128 Mb of serial flash memory via a QSPI interface.

General-purpose I/O consists of six pushbuttons, four slide switches, and five LEDs — resources leveraged in this project for status indication via the onboard RGB LED. Six Pmod expansion ports accommodate additional peripherals, and an on-board JTAG interface combined with a USB-to-UART bridge enables programming and serial monitoring without requiring external debug hardware.

Taken together, the Zybo's combination of a real-time capable dual-core processor, reconfigurable hardware logic, and a comprehensive set of onboard peripherals justified its selection as the computational platform for this flight controller. It provided the necessary performance headroom, hardware flexibility, and peripheral availability to implement all required subsystems — wireless control data reception via an FPGA-hosted AXI UART block, inertial sensor acquisition over I2C, PID computation, and motor actuation — within a single compact and accessible development board [5].

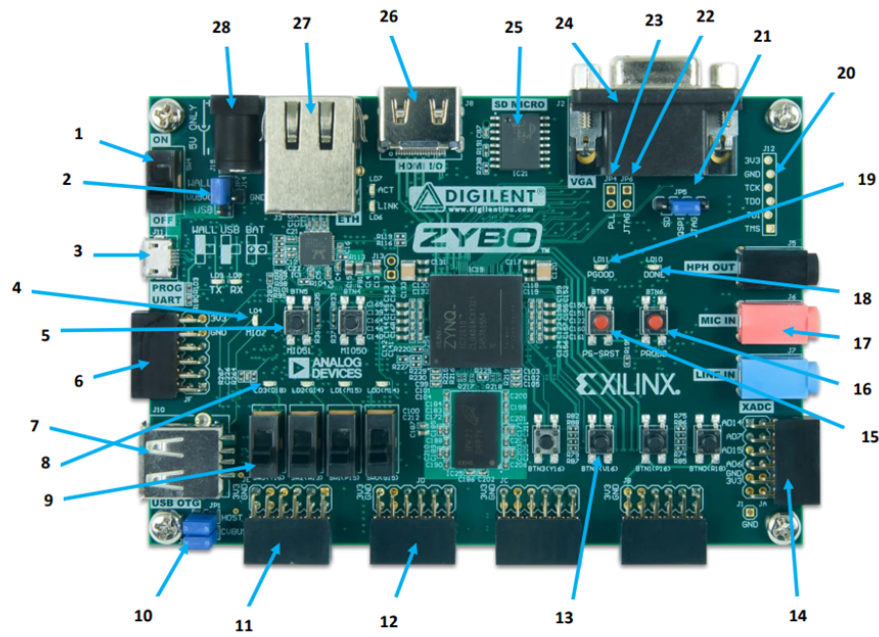


Figure 1.7: Zybo Board

Table 1.2: Zybo Board Components

Callout	Component Description	Callout	Component Description
1	Power Switch	15	Processor Reset Pushbutton
2	Power Select Jumper and battery header	16	Logic configuration reset Pushbutton
3	Shared UART/JTAG USB port	17	Audio Codec Connectors
4	MIO LED	18	Logic Configuration Done LED
5	MIO Pushbuttons (2)	19	Board Power Good LED
6	MIO Pmod	20	JTAG Port for optional external cable
7	USB OTG Connectors	21	Programming Mode Jumper
8	Logic LEDs (4)	22	Independent JTAG Mode Enable Jumper
9	Logic Slide switches (4)	23	PLL Bypass Jumper
10	USB OTG Host/Device Select Jumpers	24	VGA connector
11	Standard Pmod	25	microSD connector (Reverse side)
12	High-speed Pmods (3)	26	HDMI Sink/Source Connector
13	Logic Pushbuttons (4)	27	Ethernet RJ45 Connector
14	XADC Pmod	28	Power Jack

Zynq 7000 SoC Functional Blocks

The following figure 1.8 illustrates the functional blocks of the Zynq 7000 SoC. The PS and the PL are on separate power domains, enabling the user of these devices to power down the PL for power management if required [5].

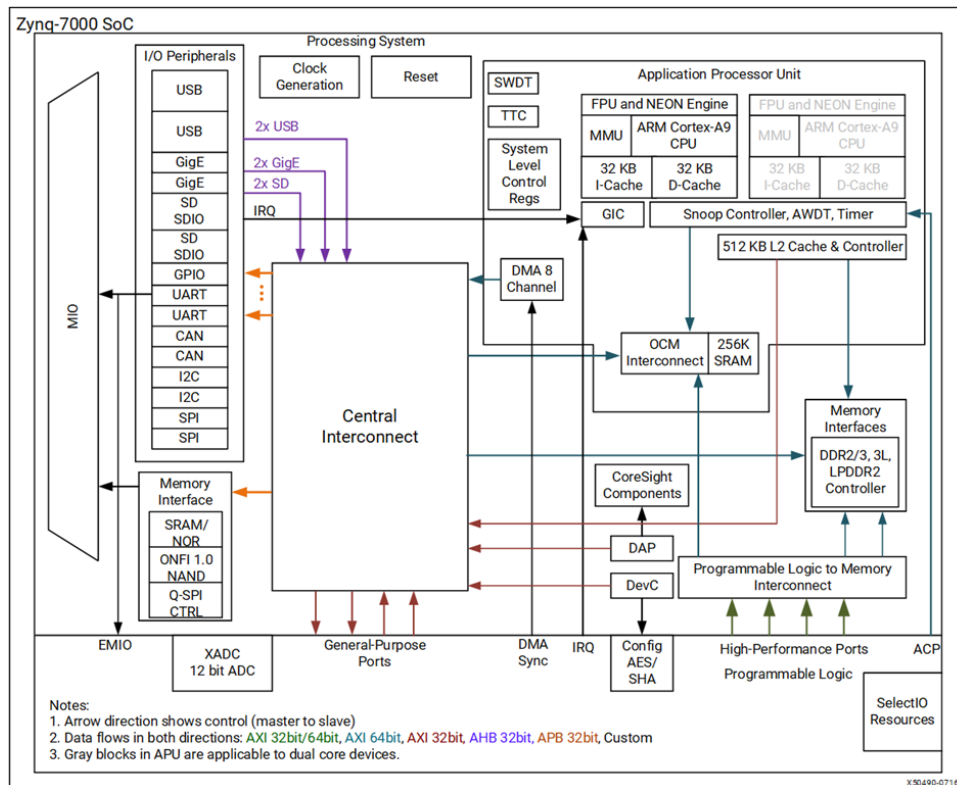


Figure 1.8: Zynq 7000 SoC Functional Blocks

The Zynq 7000 SoC is composed of the following major functional blocks:

- **Processing System (PS)**
 - Application processor unit (APU)
 - Memory interfaces
 - I/O peripherals (IOP)
 - Interconnect
- **Programmable Logic (PL)**

Power Supply

The Zybo board supports three distinct power input configurations, selectable via jumper JP7 located adjacent to the main power switch (SW4): USB bus power, an external DC wall adapter via the barrel jack (J15), and an external battery pack. This flexibility is particularly relevant for an embedded flight controller application, where operation independent of a host PC is a fundamental requirement.

When powered via USB, the board draws current from the Micro-USB JTAG/UART port (J11). While this option is convenient during development and debugging, the USB 2.0 specification limits available current to 500 mA at 5V. An idle board consumes approximately 200 mA, leaving limited headroom for demanding real-time applications. Accordingly, for deployment in a flight controller context, an external power source is the appropriate choice.

An external DC wall adapter can be connected through the barrel jack (J15) by setting JP7 to the “WALL” position. The adapter must supply between 4.5 V and 5.5 V with a minimum current rating of 2.5 A (12.5 W), using a center-positive 2.1 mm coaxial connector. Voltages exceeding 6 V risk permanent damage to the board and must be avoided. Alternatively, a battery pack may be connected

directly to the JP7 center pin and ground reference J14, provided its voltage remains within the 3.6 V to 5.5 V range — or a minimum of 4.6 V if the USB Host or HDMI source functions are active (Figure 1.9).

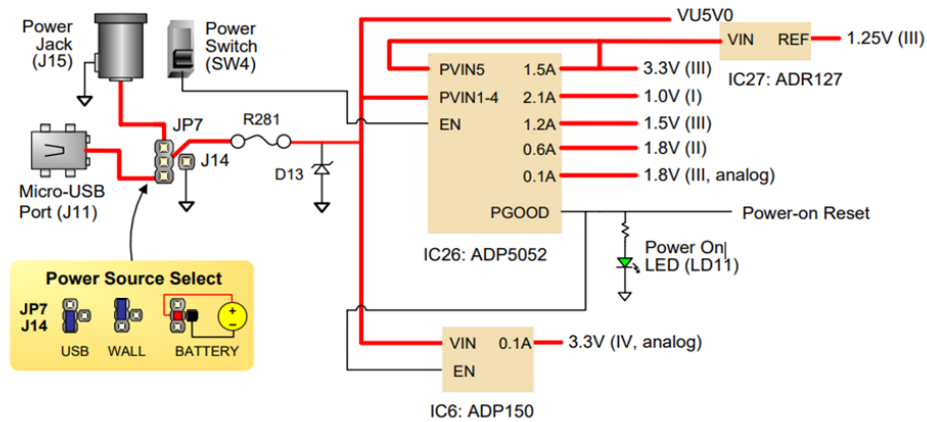


Figure 1.9: Power Supply Simplified Schematic

Onboard voltage regulation is handled by an Analog Devices ADP5052 multi-channel switching regulator (IC26), which generates the four primary supply rails required by the Zynq SoC and its peripherals. An additional ADP150 LDO (IC6) provides a dedicated 3.3 V analog rail for the audio codec, and an ADR127 precision reference (IC27) supplies a stable 1.25 V reference to the XADC analog input. Table 1.3 summarises the supply rails and their associated loads [5].

Table 1.3: Zybo onboard power supply rails.

Rail	Load	Max Current
3.3 V	FPGA I/O, USB, Ethernet, SD, Flash, HDMI	2.5 A
1.0 V	FPGA core, Ethernet core	2.5 A
1.5 V	DDR3 memory	1.2 A
1.8 V	FPGA auxiliary, Ethernet I/O, USB OTG	1.2 A
1.8 V	XADC analog	200 mA
3.3 V	Audio analog	150 mA
1.25 V	XADC precision reference	5 mA

The supply rails follow the Xilinx-recommended power-up sequencing order. Upon activation of the power switch, the 1.0 V core rail is enabled first, followed by the 1.8 V digital rail, and subsequently the 3.3 V and 1.5 V I/O rails. The 1.25 V reference and 1.8 V analog supplies ramp concurrently with the 3.3 V rail. Each channel employs a soft-start ramp of 1 to 10 ms to limit inrush current. Once all channels of the ADP5052 reach regulation, the PGOOD signal asserts, enabling the audio supply and illuminating the power indicator LED (LD11). A minimum delay of 130 ms is then observed before the Power-On Reset signal (PS_POR_B) is de-asserted, allowing the system clock (PS_CLK) to stabilise prior to processor initialisation.

This structured power sequencing ensures reliable and repeatable startup behaviour, a property of practical importance in a flight controller where an incomplete or unstable initialisation sequence could have significant safety implications.

1.6.1 Hardware/Software Partitioning Strategy

A fundamental design consideration in any Zynq-based system is the deliberate allocation of tasks between the Processing System (PS) and the Programmable Logic (PL). This partitioning decision is driven by the real-time constraints, computational nature, and timing sensitivity of each functional subsystem within the flight controller. Rather than treating the PS and PL as independent execution environments, the Zynq architecture encourages a co-design approach in which each domain handles exclusively what it is architecturally best suited for, and the boundary between the two is defined by the nature of the task rather than convenience.

Tasks that are inherently time-critical, require precise and deterministic signal timing, or must operate independently of software execution are natural candidates for implementation in the programmable logic. In this design, motor actuation falls squarely into this category: generating four independent PWM signals with microsecond-level accuracy and strict period consistency demands a level of temporal determinism that a software-driven approach cannot reliably guarantee, particularly under varying processor load conditions. Delegating this responsibility to dedicated hardware logic ensures that motor output remains stable and temporally consistent regardless of the execution state of the software running on the ARM cores. Any jitter or delay introduced at the actuation stage translates directly into mechanical instability in the drone, making hardware-level PWM generation not merely preferable but necessary.

Similarly, the acquisition of inertial data from the MPU-9265 inertial measurement unit benefits from a dedicated hardware peripheral managing the communication bus. Offloading the low-level protocol handling — including start/stop condition generation, clock stretching, and acknowledgment management — to a hardware block allows sensor reads to be initiated and completed with minimal processor involvement, freeing the ARM cores from the overhead of bit-level communication management. This is particularly relevant given that the IMU must be sampled at a consistent rate to maintain the integrity of the complementary filter and PID feedback loop.

The reception of wireless control commands transmitted by the ESP-01S module over a UART serial link is likewise assigned to the programmable logic through a dedicated AXI UART peripheral. Isolating this communication channel within the PL ensures that incoming control frames are captured and buffered independently of the processor's activity, preventing data loss during computationally intensive phases of the control loop. Furthermore, this architectural choice preserves the PS UART interfaces for their intended purposes — debug terminal output, hardware-in-the-loop communication, and host-facing diagnostics — avoiding resource contention between functionally distinct communication channels. Status indication through the onboard RGB LED is also managed through a hardware GPIO peripheral instantiated in the PL, keeping the processor free from any polling or output management duties related to system state visualisation.

Conversely, tasks that are computationally intensive, algorithmically complex, or inherently sequential are better suited to execution on the dual-core ARM Cortex-A9 processor. The flight state machine, PID control loops, sensor fusion through the complementary filter, and all debug and host-facing communication are therefore handled entirely in software on the PS. This layer benefits from the full expressiveness of a C programming environment, floating-point arithmetic, and access to the Xilinx SDK peripheral driver ecosystem. The processor is thus never burdened with signal-level timing tasks, and conversely, the programmable logic is never asked to perform decisions or computations that require contextual awareness of system state.

This partitioning strategy reflects a broader principle in heterogeneous embedded system design: the programmable logic acts as a deterministic real-time front-end that abstracts hardware-level signal acquisition and actuation away from the processor, while the processor focuses exclusively on the control intelligence of the system. The boundary between the two domains is not arbitrary — it is

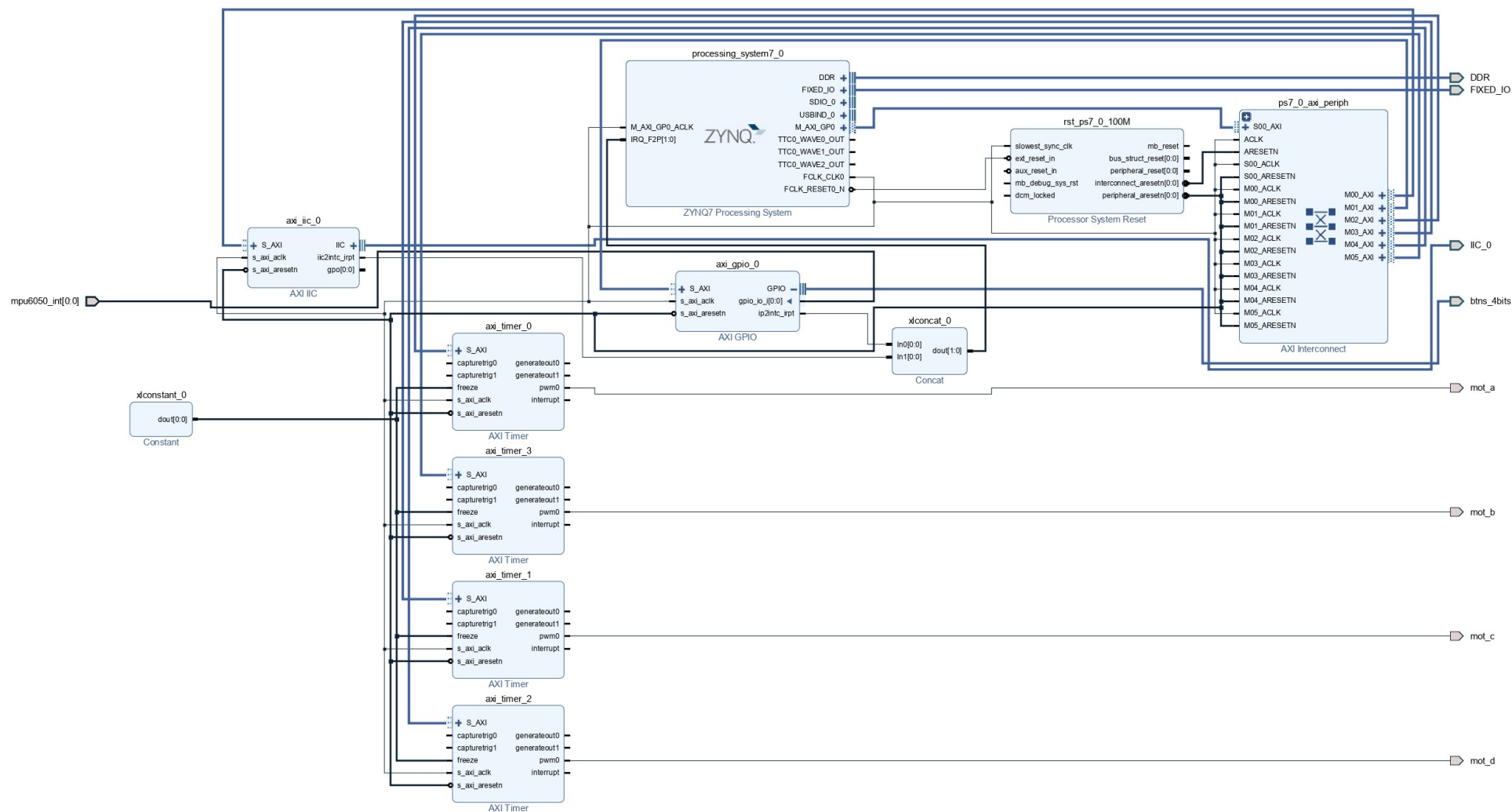


Figure 1.10: Complete Vivado IP Integrator block design for the Zynq-7010 SoC. This overview highlights the AXI Interconnect architecture bridging the Processing System (PS) with the Programmable Logic (PL) peripherals, including PWM generators for motor control and I2C/UART interfaces.

drawn precisely where the nature of the task transitions from timing-sensitive hardware behaviour to context-aware software decision-making. The result (Figure 1.10) is a clean and maintainable architecture in which neither domain is over-solicited, latency is minimised at every stage of the signal chain, and the overall system exhibits the predictability and responsiveness that a safety-critical application such as a flight controller demands[16].¹

1.6.2 Peripheral Configuration: PWM, I²C and UART

PWM Generation Architecture: PS TTC versus PL AXI Timer

While the Processing System (PS) of the Zynq-7000 SoC integrates hard-wired Triple Timer Counters (TTC), implementing a soft AXI Timer within the Programmable Logic (PL) yields superior deterministic performance for high-dynamic embedded applications, such as UAV flight controllers. According to the device technical reference manual [5], the built-in TTC modules are structurally limited to 16-bit prescalers and counters.

In the context of high-frequency motor control protocols (e.g., SimonK 30A ESC), this 16-bit ceiling restricts duty-cycle resolution and introduces quantization error when translating fine-grained Proportional-Integral-Derivative (PID) controller outputs into physical signals. Furthermore, the TTC is inherently synchronized to the internal PS bus clock (CPU_1x). This architectural dependency theoretically exposes critical motor control signals to nanosecond-level timing jitter induced by concurrent software execution, interrupts, or operating system load variations on the ARM Cortex-A9 cores.

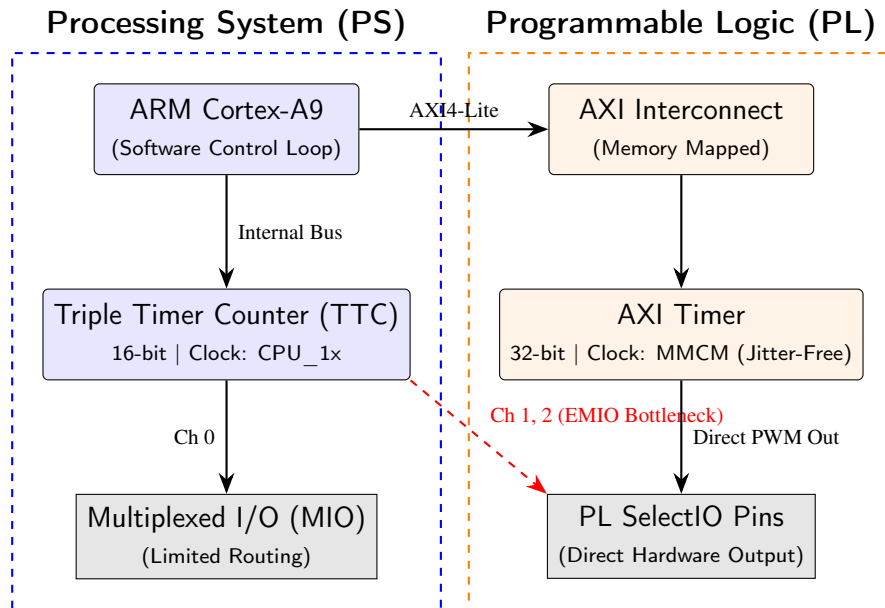


Figure 1.11: Architectural comparison of PWM generation pathways. The AXI Timer provides a direct, 32-bit, jitter-free route to the physical SelectIO pins, avoiding the 16-bit TTC limitations and EMIO routing bottlenecks.

Conversely, instantiating an AXI Timer IP within the PL addresses these limitations by providing full

¹It is worth noting that the Zynq XC7Z010, at the heart of the Zybo board, belongs to the Zynq-7000 SoC family and represents the smallest member of that product line, particularly in terms of programmable logic resources. Its PL fabric is more constrained in capacity compared to higher-tier members of the same family such as the Z-7020 or Z-7045. Nevertheless, when the PS/PL boundary is drawn with care and the logic fabric is allocated exclusively to hardware tasks that genuinely require it — as is the case in the partitioning strategy described above — the Z-7010 demonstrates that resource efficiency and architectural rigour can compensate for raw capacity. Within the scope of a quadrotor flight controller, the combination of a 650 MHz dual-core processor and a well-utilised FPGA fabric constitutes a capable and sustainable embedded platform, provided that the design avoids unnecessary logic inflation and respects the constraints of the device.

32-bit counter resolution, enabling sub-microsecond granularity for the PWM duty cycle. Because the PL-based timer operates on a dedicated, highly stable hardware clock synthesized by a Mixed-Mode Clock Manager (MMCM), it effectively decouples the real-time signal generation from the processor domain, ensuring strict hardware determinism.

Physical routing constraints (Figure 1.12) further validate the necessity of a PL-centric approach. As documented in the system's I/O architecture [5], the limited availability of dedicated Multiplexed I/O (MIO) pins dictates that secondary and tertiary TTC signals must be routed via the Extended MIO (EMIO)—which physically traverses the PL fabric to reach external pins (see Figure 1.11). Therefore, natively generating the multi-channel PWM signals within the PL via an AXI Timer not only circumvents the MIO multiplexing bottleneck but leverages the highly configurable High-Range (HR) SelectIO pins, ensuring scalable, jitter-free connectivity directly to the electronic speed controllers (ESCs).

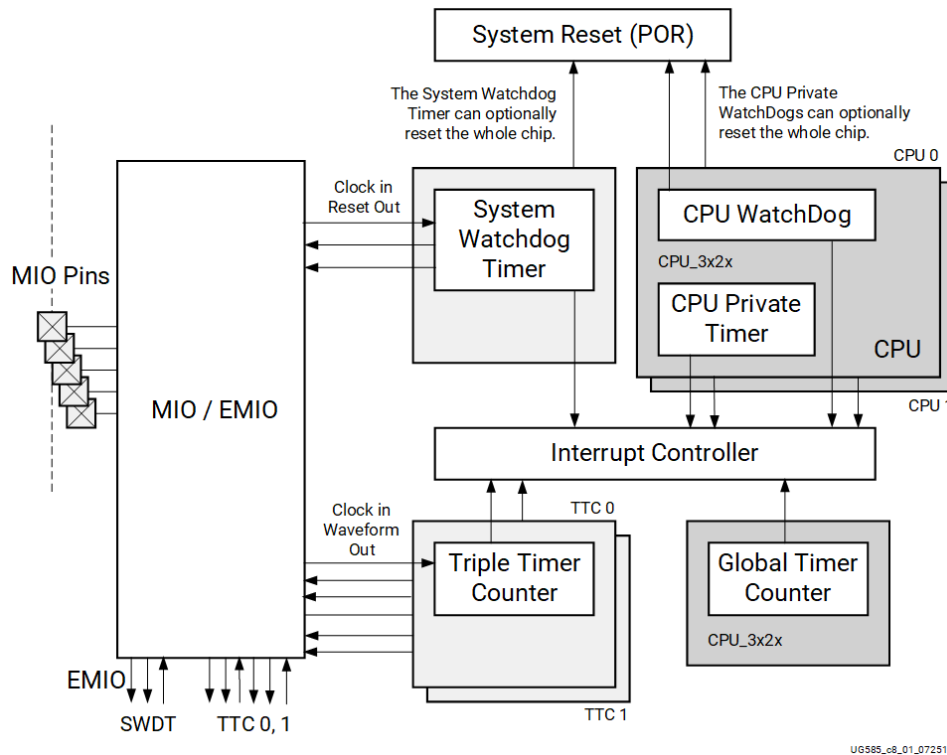


Figure 1.12: Zynq-7000 signal routing pathways demonstrating the physical limitations of the Multiplexed I/O (MIO) forcing secondary TTC channels through the Extended MIO (EMIO).

I2C Communication Architecture: PS I2C versus PL AXI IIC

Similar to the constraints observed in timer instantiation, relying on the Zynq-7000 Processing System's built-in I2C controllers presents several bottlenecks for high-throughput, multi-sensor embedded applications. According to the device documentation [5], the PS features two hard-wired I2C controllers, each restricted to a 16-byte FIFO. In modern UAV flight controllers, reading high-speed burst data from an Inertial Measurement Unit (IMU) typically exceeds this 16-byte payload. Consequently, the PS I2C controller must frequently interrupt the ARM Cortex-A9 cores to service the shallow FIFO, introducing significant software overhead and latency into the real-time control loop.

Furthermore, the PS I2C Serial Clock (SCL) frequency is derived by dividing the internal CPU_1x bus clock. The mathematical constraints of this division mean that achieving precise standard SCL frequencies (e.g., 400 kHz Fast Mode) is occasionally impossible depending on the active CPU clock rate, potentially leading to bus synchronization issues. The system is also physically restricted to

two buses, which poses a critical limitation when integrating multiple redundant sensors that share identical, hard-coded hardware addresses.

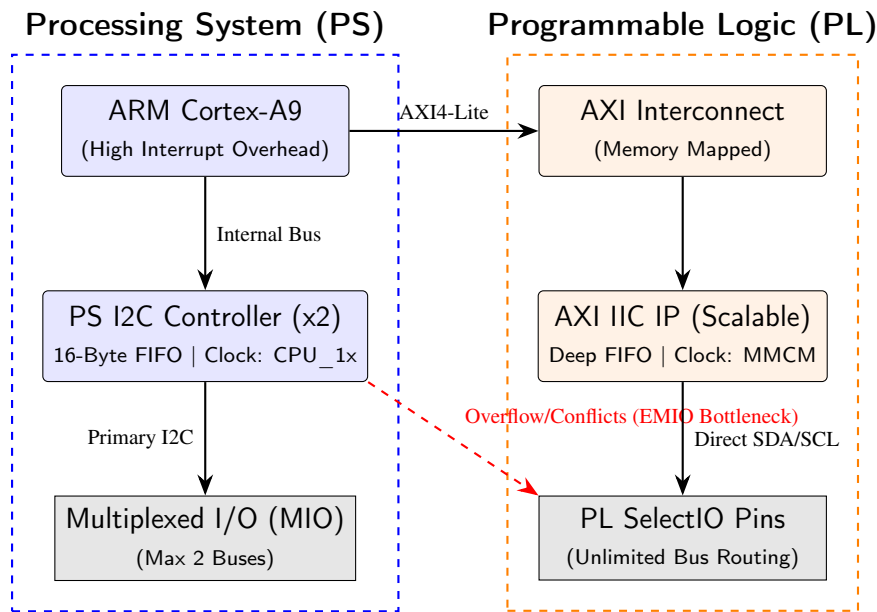


Figure 1.13: Architectural comparison of I2C bus instantiation. The PL-based AXI IIC mitigates the 16-byte FIFO interrupt bottleneck, allows for unlimited bus instantiation to avoid address conflicts, and provides precise MMCM-backed clocking.

Instantiating one or more AXI IIC soft-IP blocks within the Programmable Logic (PL) resolves these systemic limitations. As illustrated in Figure 1.13, the AXI IIC provides a configurable, deep FIFO that mitigates processor interrupt overhead during large burst reads. Furthermore, the AXI IIC operates on an independent, hardware-synthesized Mixed-Mode Clock Manager (MMCM) signal, ensuring exact SCL frequency generation regardless of the CPU state. Architecturally, offloading the I2C control to the PL allows designers to instantiate an arbitrary number of independent I2C buses directly routed to the High-Range (HR) SelectIO pins. This easily bypasses the MIO routing constraints and eliminates the inherent sensor address conflicts found on shared, hardware-limited PS buses[5].

Asynchronous Serial Reception: PS UART versus PL AXI UART

For handling asynchronous serial communication—specifically receiving high-speed telemetry and command data from an external ESP-01S Wi-Fi module—the hardware architecture of the UART controller dictates the system’s interrupt overhead and data integrity. The Zynq-7000 Processing System incorporates two hard-wired UART controllers, which, according to the device’s technical reference manual [5], are constrained by a fixed 64-byte receive (Rx) FIFO. Because Wi-Fi payloads typically arrive in dense, high-speed bursts (often exceeding several hundred bytes per packet), this shallow 64-byte buffer necessitates aggressive, high-frequency interrupt servicing by the ARM Cortex-A9 processor to prevent receiver overflow errors (ROVR).

Furthermore, the PS UART baud rate generator derives its bit-period clock by dividing a shared, system-level UART_Ref_Clk. The mathematical limitations of this fixed-divider architecture introduce measurable quantization errors at higher standard baud rates (e.g., 115,200 baud), which can cause frame misalignment and data corruption during extended asynchronous bursts.

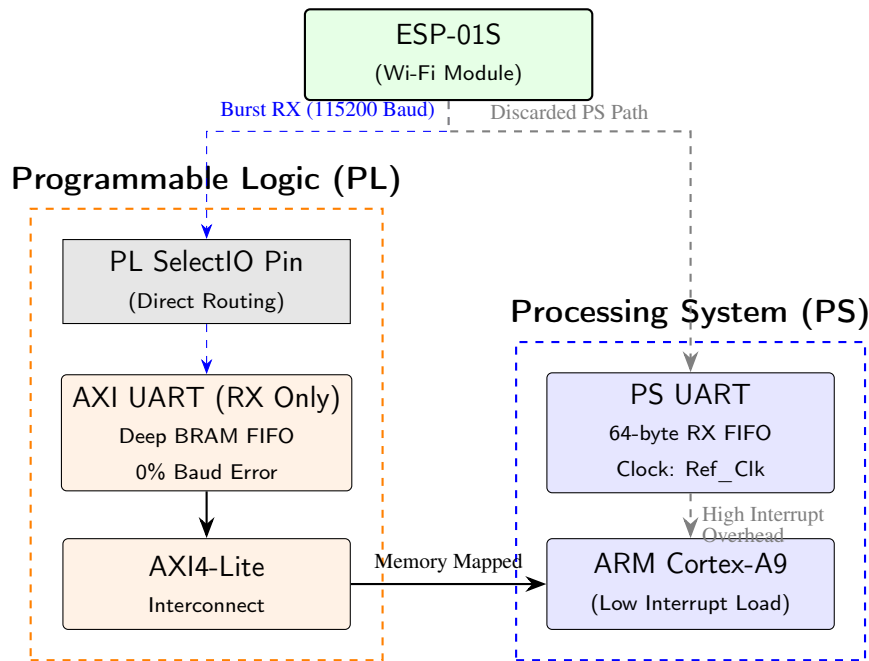


Figure 1.14: Data reception architecture for the ESP-01S Wi-Fi module. Utilizing a PL-based AXI UART dedicated strictly to reception allows for a deep, BRAM-backed FIFO that buffers large network packets, significantly reducing CPU interrupt overhead compared to the hard-wired 64-byte PS UART limit.

To guarantee data integrity and decouple the real-time flight control loop from networking overhead, the reception path is implemented entirely within the PL using an AXI UART intellectual property (IP) block. As shown in Figure 1.14, this architecture connects the ESP-01S transmission line directly to a PL SelectIO pin. The AXI UART is configured with an expanded, Block RAM-backed receiver FIFO, capable of caching complete Wi-Fi message packets. Furthermore, driving the IP with a dedicated PL Mixed-Mode Clock Manager (MMCM) synthesizes a highly precise baud clock, eliminating the fractional sampling errors inherent to the PS clocking architecture. The ARM Cortex-A9 is only interrupted when a complete packet is ready for extraction via the memory-mapped AXI4-Lite bus, ensuring maximum CPU availability for critical flight dynamics calculations[16]. Here is a humanized, elaborated, and reformulated version of your subsection:

FPGA Resource Utilization and Power Analysis

One of the central questions when adopting a hardware-offloading strategy is whether the performance gains justify the cost in silicon area and power budget. To answer this concretely, a full post-implementation analysis was carried out in the Vivado Design Suite following the synthesis and place-and-route of the complete design. The results are not only encouraging — they make a compelling case for the architectural choices made throughout this project.

Resource Utilization. The combined logic footprint of the three AXI-based peripherals — the AXI Timer driving the PWM outputs, the AXI IIC handling sensor communication, and the AXI UART managing the telemetry link — was found to be remarkably modest. As summarized in Figure 1.15, the entire peripheral subsystem occupies just 2,397 Look-Up Tables (LUTs) and 2,261 Flip-Flops (FFs) on the Zynq-7010 fabric, corresponding to roughly 13.6% and 6.4% of the device’s available logic resources, respectively. The LUTRAM usage stands at only 82 entries (1.37%), and IO utilization is limited to 9 pins (9%).

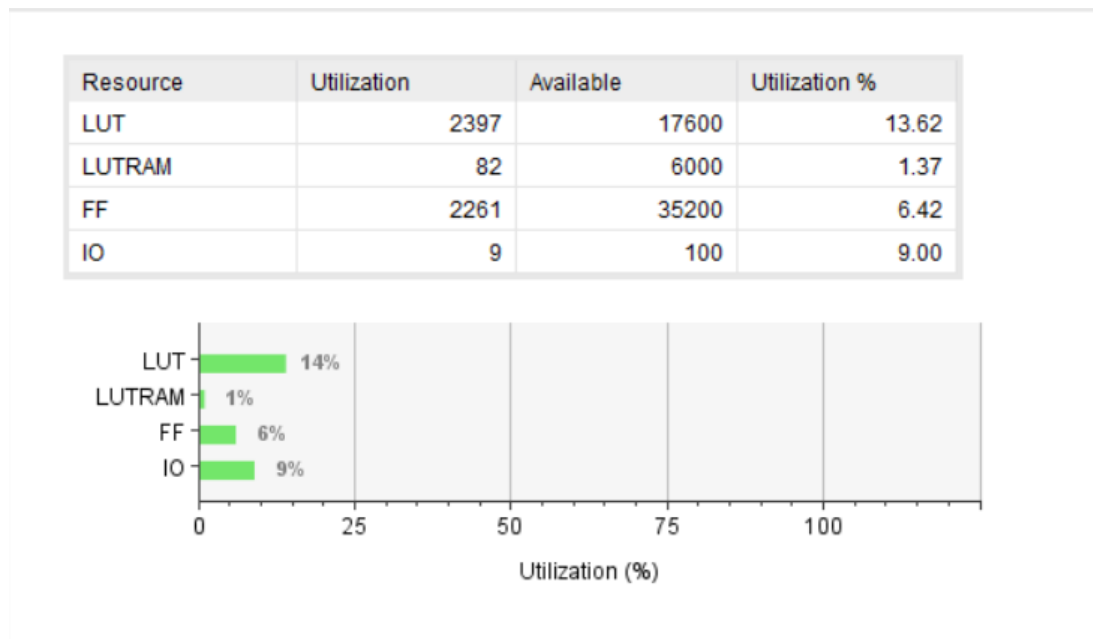


Figure 1.15: Post-implementation resource utilization summary on the Zynq-7010, showing the lightweight footprint of the PL-based peripheral architecture.

To put these numbers in perspective: more than 85% of the FPGA's reconfigurable fabric remains completely untouched. This headroom is not incidental — it is a deliberate design outcome. It means the platform retains the capacity to accommodate future hardware accelerators without any architectural rework. Candidate extensions might include a hardware-implemented Kalman filter for attitude estimation, a custom motor-mixing matrix operating at sub-microsecond latency, or even a neural inference engine for onboard adaptive control. The current design, in other words, is not a ceiling — it is a foundation.

Physical Placement and Routing. Beyond raw numbers, the quality of the implementation matters. The Vivado device floorplan, shown in Figure 1.16, offers a spatial view of how the synthesized logic was distributed across the FPGA die. The cyan-colored cells — representing the instantiated LUTs and FFs — are spread cleanly across the central fabric columns, with no signs of routing congestion or hotspot clustering. The SelectIO pads interfacing with the physical peripherals are correctly anchored along the device boundary, and the AXI interconnect wiring follows predictable, unobstructed paths between the Processing System and the custom IP blocks.

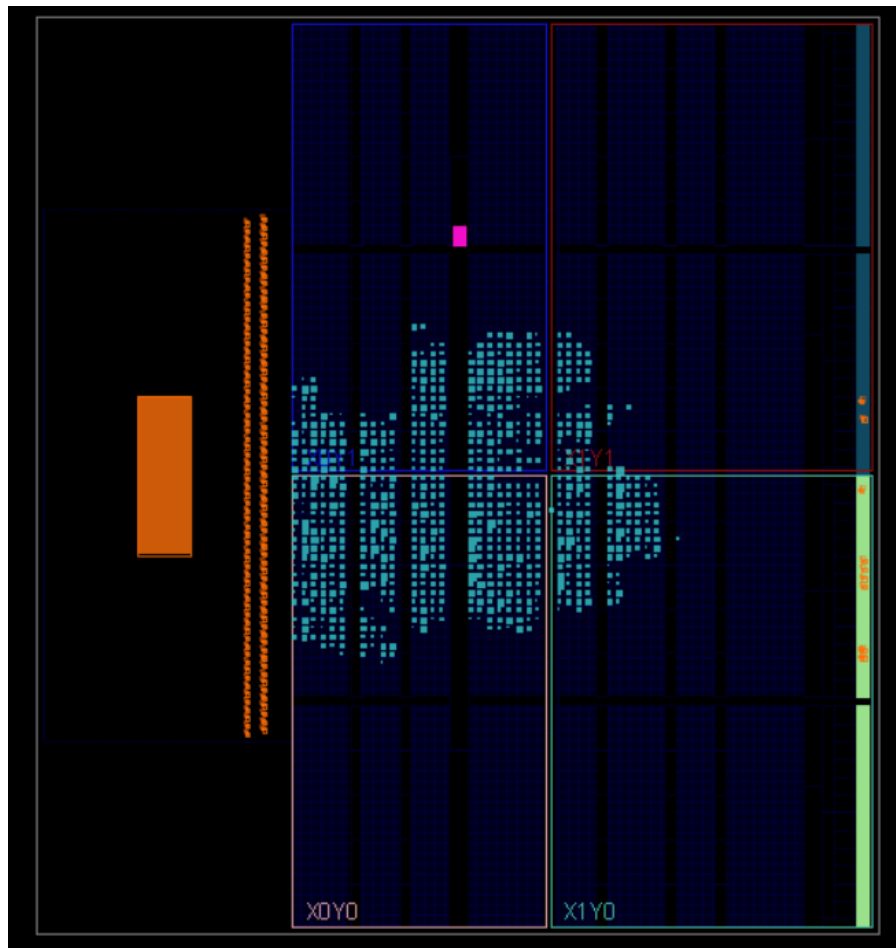


Figure 1.16: Vivado device floorplan showing the physical placement and routing of the AXI-based peripherals within the Programmable Logic, with cyan cells indicating implemented logic resources.

This clean placement is significant from a signal integrity standpoint. In high-frequency switching applications such as PWM generation, poorly routed logic can introduce timing skew or glitches that corrupt actuator commands. The routing quality observed here, with short interconnect paths and balanced fanout, confirms that the implemented design will behave deterministically under operational conditions — a non-negotiable requirement for flight-critical hardware.

Power Consumption. Perhaps the most practically relevant metric for an embedded UAV platform is power efficiency. Every milliwatt drawn by the flight controller is a milliwatt unavailable to the propulsion system, and thermal dissipation must remain within safe bounds during extended flight missions. The on-chip power estimation report, presented in Figure 1.17, breaks down the 1.705 W total consumption in a revealing way.

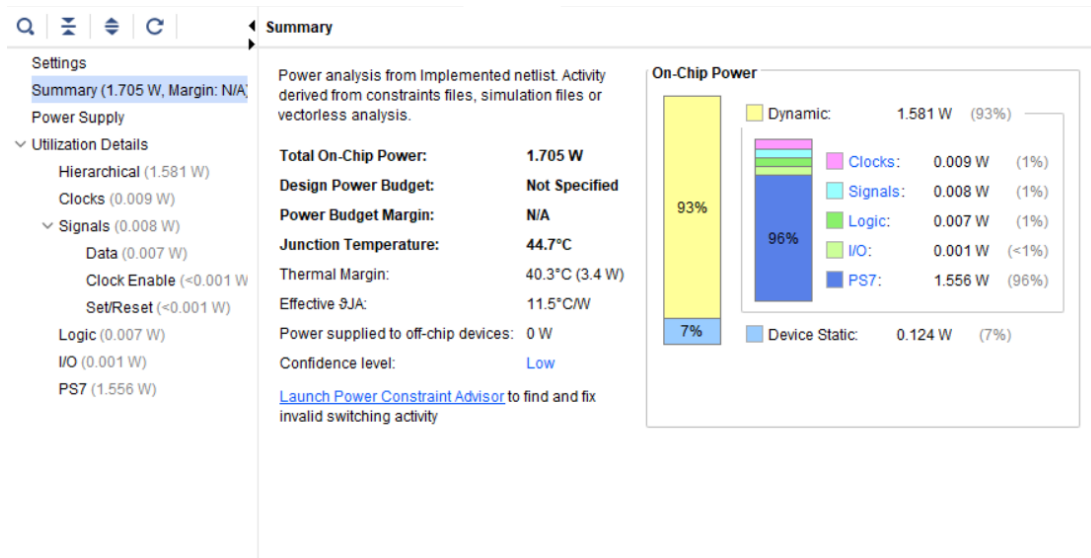


Figure 1.17: On-chip power estimation report from Vivado, illustrating the dominant contribution of the Processing System (PS7) and the negligible overhead of the hardware-offloaded peripherals.

The dominant contributor — accounting for 1.556 W, or 96% of dynamic power — is the PS7 block, i.e., the ARM Cortex-A9 dual-core processor and its associated memory subsystems running the flight management software. This is expected and unsurprising: a general-purpose processor executing complex flight dynamics and sensor fusion loops will always carry a substantial base power cost.

What is truly noteworthy, however, is the power budget attributed to the custom PL logic. The combinational logic cells consume approximately 0.007 W, signal switching accounts for 0.008 W, and clock distribution adds another 0.009 W. Together, the entire hardware peripheral subsystem — responsible for generating all PWM control signals, managing all I²C sensor transactions, and handling all UART telemetry exchanges — draws less than 25 mW in total. This figure underscores one of the key architectural advantages of offloading to reconfigurable logic: tasks that would require continuous processor interrupts, DMA overhead, and software timing loops are instead handled by dedicated hardware at a fraction of the energy cost.

The estimated junction temperature of 44.7°C, with a thermal margin of 40.3°C before reaching the device's rated limit, further confirms that the design operates well within safe thermal bounds even under full activity.

Summary. Taken together, these post-implementation results validate the core premise of the hardware offloading strategy. The custom AXI peripheral architecture delivers strict timing determinism for PWM, I²C, and UART interfaces while consuming a negligible share of the device's logic resources and power budget. The Processing System is freed from the burden of bit-banging and interrupt-driven peripheral management, and the FPGA fabric remains largely available for future expansion. This combination of low overhead, clean routing, and thermal headroom confirms that the proposed flight controller architecture is not only functionally correct, but genuinely well-suited for deployment in resource-constrained, power-sensitive UAV applications.

1.7 Conclusion

This chapter has laid the theoretical and architectural groundwork for the flight controller developed in this project. Beginning with a comparative analysis of existing aerial platforms, the quadrotor's equations of motion were derived from first principles and subsequently linearised about the hover

trim condition using small perturbation theory, yielding a state-space model of the form $\dot{\mathbf{X}} = \mathbf{A}\mathbf{X} + \mathbf{B}\mathbf{U}$ suitable for control law synthesis. Structural considerations — arm length, center of gravity placement, motor orientation, and autopilot mounting — were shown to be direct prerequisites for the validity of this model rather than purely practical choices.

On the embedded side, the Zynq-7010 SoC's tightly coupled PS/PL architecture was shown to be well-matched to the deterministic demands of a flight controller. A principled hardware/software partitioning strategy was defined, assigning time-critical tasks — PWM actuation, sensor interfacing, and UART command reception — to dedicated AXI peripherals in the programmable logic, while reserving the ARM Cortex-A9 processor for the PID control loops and sensor fusion. Together, the physical model and the embedded architecture form the consistent design framework upon which the subsequent chapters build.

Chapter 2

Implementation and Experimental Results

2.1 Introduction

With the theoretical framework and system architecture established in the preceding chapter, this chapter details the practical implementation of the quad-rotor flight controller and the experimental results obtained at each stage of its development. Following a bottom-up approach, the implementation progresses from the low-level hardware interfaces to the higher-level control algorithms, each subsystem being validated before the next is introduced.

The chapter begins with the propulsion system and wireless control link, covering PWM signal generation, ESC interfacing, and the ESP-01S Wi-Fi integration. System identification then follows, with the experimental determination of the quad-rotor's moments of inertia providing the physical parameters required by the control laws. Sensor interfacing and calibration are subsequently addressed, establishing a reliable state estimation pipeline through the MPU6050 and HMC5883L sensor suite. The fused attitude estimates produced by the Kalman filter then serve as the feedback signal for the PID controller, whose gains are determined through a Hardware-in-the-Loop (HIL) simulation environment. The chapter concludes with the analysis of the flight test results, validating the complete system against the performance requirements defined in Chapter 1.

2.2 Actuator Control and Telemetry

2.2.1 PWM Signal Generation (PL Implementation)

The flight controller regulates the amount of power delivered to the propulsion system via Pulse Width Modulation (PWM). It controls the average power supplied to the load by varying the width of the pulse signal. The duty cycle (D) represents the ratio of the pulse width (T_{on}) to the total period (T_{total}). An increase in the duty cycle results in a higher average voltage (V_{av}), as defined by equations (2.1) and (2.2):

$$D = \frac{T_{on}}{T_{total}} \times 100\% \quad (2.1)$$

$$V_{av} = D \times V_{on} \quad (2.2)$$

The width of the PWM pulse signals (figure 2.1) delivered by the flight controller vary between **1ms** and **2ms** within a typical 20ms period (50Hz), to ensure compatibility with standard Electronic Speed Controllers (ESCs).

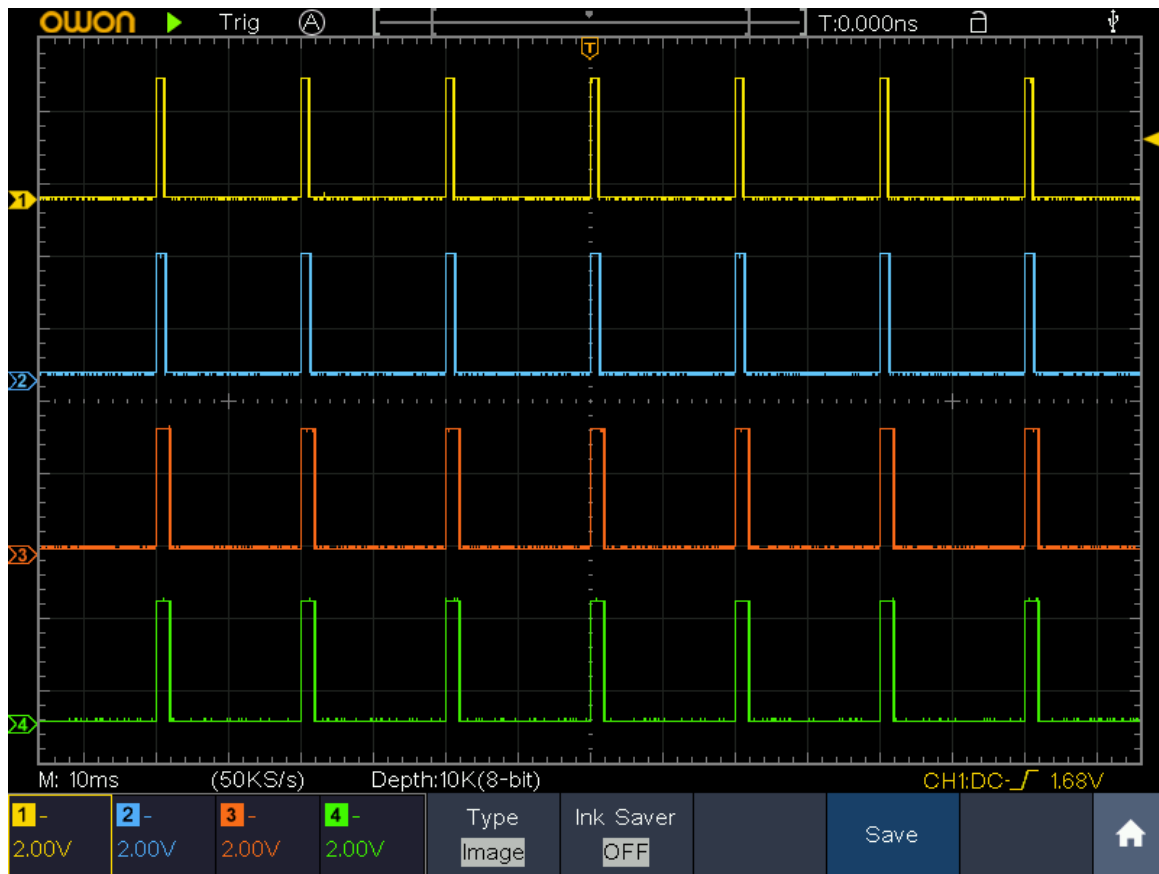


Figure 2.1: Oscilloscope observations of PWM Signals at different duty cycles

The PWM generation is implemented using four AXI Timer IP cores mapped in the processor's memory space (Seen in figure 1.10). The flight controller software writes values to the motors using the function *update_motor*.

Algorithmic Flow of *update_motor* function

- The function receives as parameters a motor identifier and a throttle percentage. It applies clamping to ensure the percentage is between 0% and 100% and prevent overflow:

```
if (percent < 0) percent = 0;
if (percent > 100) percent = 100;
```

- The percentage is converted to a signal in the pulse-width range of 1.0 ms to 2.0 ms using the equation:

```
uint32_t high_time = PULSE_MIN_1MS + (percent * 1000);
```

- The calculated value is written to the AXI Timer's Load Register via the AXI4-Lite bus:

```
XTmrCtr_WriteReg(baseaddr, 1, XTC_TLR_OFFSET, high_time);
```

- The `XTC_CSR_LOAD_MASK` bit is triggered for both Timer 0 and Timer 1, acting as a hardware strobe that instantly latches the period and duty cycle values into their respective active counters:

```
XTmrCtr_WriteReg(baseaddr, 0, XTC_TCSR_OFFSET, XTC_CSR_LOAD_MASK);
XTmrCtr_WriteReg(baseaddr, 1, XTC_TCSR_OFFSET, XTC_CSR_LOAD_MASK);
```


- The timers 0 and 1 are initialized in PWM generation mode by writing the configuration mask 0x206 to their Control Status Registers. This enables the PWM logic and links the timers to the physical output pins.

```
XTmrCtr_WriteReg(baseaddr, 0, XTC_TCSR_OFFSET, 0x206);
XTmrCtr_WriteReg(baseaddr, 1, XTC_TCSR_OFFSET, 0x206);
```

The four output PWM signals are then generated according to the required duty cycle and fed directly to the propulsion system.

2.2.2 Propulsion System, ESC Interfacing, and Power Distribution

The propulsion system is a fundamental component of the quad-rotor, as it directly dictates its main performance metrics such as the hovering time, flying speed and distance, as well as its payload capacity. It includes the motors, propellers and electronic speed controllers (ESCs). Choosing the right components and ensuring their mutual compatibility is crucial for the whole system to function properly, avoiding failures mid-air and safety concerns.

ESCs and Brushless DC Motors

For the propulsion system to function, the pulse width modulated (PWM) signal generated by the flight controller is first sent to the electronic speed controllers (ESCs). These components serve as an interface between the motors and the power source, ensuring smooth acceleration and deceleration while preventing voltage and current fluctuation that could cause damage to the system.

The relationship between the throttle command σ received by the ESC, the battery output U_b and the resulting thrust and moments produced can be modeled by 2.2. After receiving the throttle command, the ESC generates an equivalent voltage U_m :

$$U_m = \sigma U_b \quad (2.3)$$

This voltage determines the steady-state angular velocity ω_{ss} through the motor constant C_R and the constant ω_b representing the offset:

$$\omega_{ss} = C_R \sigma + \omega_b \quad (2.4)$$

When given a throttle command, the motor achieves the steady-state speed ω_{ss} after a time constant T_m . The dynamics of a brushless motor can be simplified as a first order low pass filter and its transfer function expressed as :

$$\omega = \frac{1}{T_ms + 1} \omega_{ss} = \frac{1}{T_ms + 1} (C_R \sigma + \omega_b) \quad (2.5)$$

Where ω is the motor speed. Finally, the aerodynamic forces are determined from this rotational speed, where the thrust and moments are proportional to ω^2 through the coefficients c_T and c_M respectively.[19]

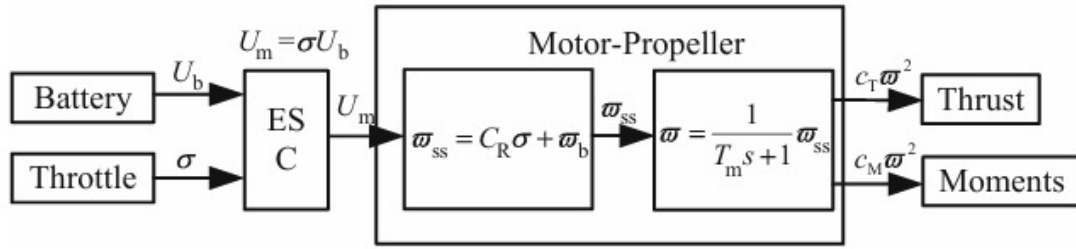


Figure 2.2: Signal transmission of the propulsor model [19]

Four single **SimonK 30A ESCs** are used in this project, powered by a 11.1V 3S LiPo battery. Each ESC receives a signal from the flight controller and connects to a brushless motor via three wires. It controls the motor by alternating currents to its stator poles in a specific precisely timed sequence, creating the magnetic field that pushes and pulls on the motor's magnets, causing it to spin.[8]



(a) SimonK 30A ESC



(b) A2212-1000KV Brushless Motor



(c) 1045 Propellers

Figure 2.3: Propulsion System Components

The ESCs main components are a **microcontroller unit (MCU)** that stores the ESC firmware, a **low dropout voltage regulator (LDO)**, **MOSFETs** that drive the motors by switching power on and off thousands of times per second, the **gate drivers** that drive the MOSFETs, **filtering capacitors** and a **current sensor** that can be used to track the battery state [13].

The ESCs used are also equipped with a **Battery Eliminator Circuit (BEC)** that provides a regulated 5V power supply to the flight controller, eliminating the need for a separate battery (figure 2.4).

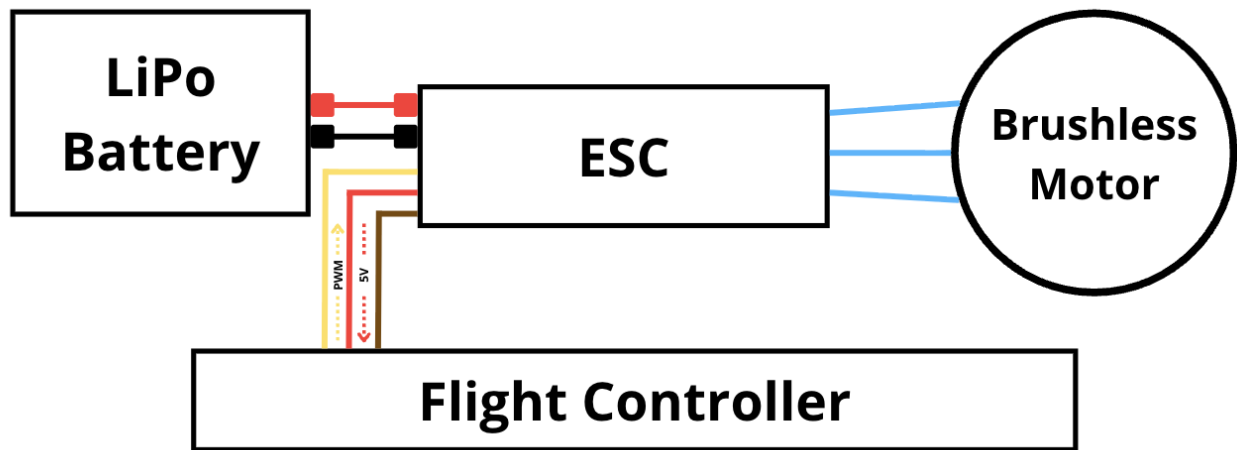


Figure 2.4: Propulsion System Wiring

Connected to each one of the ESCs is a **A2212-1000KV Brushless Motor**. These motors convert the electrical energy provided into mechanical energy. Their precision in control makes them especially suited for quad-rotor drones, the drone's attitude being controlled by precisely controlling the rotational speed of each rotor.

While similar in their core function to brush motors, brushless motors eliminate the need for the brushes that use physical contact to transfer current from one set of coils to another, causing noise, friction, a shortened operating life and limits in speed and efficiency [?].

On a 3S LiPo battery of 11.1V, the 1000KV Brushless Motors can spin at 11000 to 12000 RPM without load. This speed is however reduced under load when combined with the propellers. With the addition of four propellers (25.4 cm diameter, 11.43 cm pitch), the propulsion system of the quad-rotor is complete, and almost ready to fly. To safely do so, a calibration protocol is required.

ESC Calibration

Once the propulsion system is physically assembled, a calibration protocol is mandatory to ensure that the flight controller and the ESCs are perfectly synchronized and all four motors respond identically to throttle inputs. This is achieved by mapping the flight controller's 1000 μ s to 2000 μ s PWM range to the ESCs minimum and maximum power outputs.

During the calibration process, a number of beeps is emitted by the ESCs by applying high-frequency electrical pulses to the motor windings, inducing microscopic vibrations in the motor's rotor and vibrating the surrounding air.

The calibration is done following these steps:

1. The battery is disconnected.
2. The throttle is set at maximum.
3. The battery is connected.
4. With the throttle still at maximum, the battery is disconnected then reconnected.
5. After the ESCs emit a number of beeps corresponding to the number of battery cells with two additional beeps, the throttle is set at minimum.
6. The ESCs emit a long musical tone meaning the calibration is successfully complete.

Once the ESC calibration is complete, the propulsion system is fully functional [2].

2.2.3 Wireless Control

The drone controller is the primary command link of the quad-rotor, as it represents the only channel of communication between it and the pilot. It allows to send real time signals to the flight controller, guiding its movements and altitude.

Several options can be implemented for the wireless command transmission, the main ones being Radio Frequency, Wi-Fi and Bluetooth.

In this project, the **ESP-01S Wi-Fi module** 2.5 was selected for its easy integration via a simple Universal Asynchronous Receiver-Transmitter (UART) interface and its ground station compatibility. Indeed, it allows the quad-rotor to be controlled directly from a computer or a smart-phone via a custom graphical user interface (GUI), removing the need for a dedicated RC transmitter.

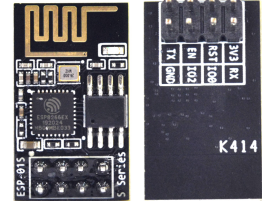


Figure 2.5: ESP-01S Module

However, this choice presents its own drawbacks, such as higher latency compared to RC transmitters and limited range, generally restricted to 30 to 50 meters.

The ESP-01S remains a very interesting option for adding wireless connectivity to the drone. Its core processor ESP8266 has a complete and self contained network function, which can be used independently. However, in this case, it will be used in slave mode, as a Wi-Fi adapter for the flight controller through a UART interface [1].

A set of AT commands, described in table 2.1, is used to configure and communicate with the Wi-Fi module, transmitted from the flight controller over UART.

Command	Name	Description / Purpose in Project
AT	Test	Verifies UART communication between the flight controller and ESP-01S.
AT+RST	Reset	Restarts the module to ensure a clean state before configuration.
AT+CWMODE=2	Wi-Fi Mode	Sets the module as an access Point.
AT+CWSAP=...	AP Config	Sets the SSID and password for the drone's Wi-Fi network.
AT+CIPMUX=1	Multi-IP	Enables multiple connections, required for stable UDP/TCP server operations.
AT+CIPSTART=...	Start UDP	Opens a UDP listener on port 4210 . The "0.0.0.0" allows any remote IP to send commands.
AT+CIFSR	Local IP	Retrieves the module's IP address to verify the network is active.
+IPD, ...	Data In	Indicates incoming network data containing pilot commands.

Table 2.1: AT commands utilized for ESP-01S communication and configuration [6].

To interface with the quad-rotor, a mobile app (RoboRemoDemo) is used as a ground control station, that allows for the development of a graphical user interface (GUI) consisting of joysticks and buttons, which transmit raw text commands over a UDP socket to the ESP-01S.

Once the wireless communication between the quad-rotor and pilot is set, and the propulsion system is properly assembled and synchronised, the physical platform is complete. However, to achieve a stable flight, the flight controller requires an accurate mathematical model of the vehicle's dynamics.

2.3 System Identification

2.3.1 Determination of Moments of Inertia

The propulsion system and wireless communication being complete, the next required step is to determine the quad-rotor's physical properties, in particular its moments of inertia.

The moments of inertia (I_{xx}, I_{yy}, I_{zz}) represent the quad-rotor's resistance to angular acceleration around its three axes. An accurate identification of these parameters is crucial for the control laws, as the torque required to rotate the vehicle is directly proportional to these values.

Due to the quad-rotor's complex geometry and the use of different materials with varying densities, calculating the inertia purely by sum-of-mass is prone to significant error. Instead, we utilized the **Torsional Pendulum Method** as seen in 2.6.



Figure 2.6: Experimental setup for the torsional pendulum method using the quad-rotor.



Figure 2.7: Comparative setup for torsion constant calibration using a reference cylinder.

To conduct this experiment, the quad-rotor is suspended from its center of mass. By rotating it slightly and recording the time taken for a number of oscillations, the period $T_{quad-rotor}$ is determined. The moment of inertia of a specific axis can be calculated as:

$$I_{axis} = \kappa_{quad-rotor} \left(\frac{T_{quad-rotor}}{2\pi} \right)^2 \quad (2.6)$$

In this case, the string's torsion constant $\kappa_{quad-rotor}$ is unknown. In order to complete the calculations, a second identical experiment (figure 2.7) is required using a cylinder of known mass ($m_{cyl} = 0.88kg$) and radius ($r_{cyl} = 50mm$), for which the theoretical moment of inertia I_{cyl} is calculated:

$$I_{cyl} = \frac{1}{2} m_{cyl} r_{cyl}^2 = 1.1 \times 10^{-3} \text{ kg} \cdot \text{m}^2 \quad (2.7)$$

By suspending this cylinder and measuring its period of oscillation ($T_{cyl} = 9.33s$), the string's torsion constant κ_{cyl} can be determined:

$$\kappa_{cyl} = \frac{4\pi^2 I_{cyl}}{T_{cyl}^2} = \frac{4\pi^2 (1.1 \times 10^{-3})}{9.33^2} = 4.98 \times 10^{-4} \text{ N} \cdot \text{m/rad} \quad (2.8)$$

However, the quad-rotor and cylinder being suspended from different string lengths, the following equation is used to calculate $\kappa_{quad-rotor}$:

$$\kappa_{quad-rotor} = \kappa_{cyl} \frac{L_{cyl}}{L_{quad}} = (4.989 \times 10^{-4}) \frac{38}{31.5} = 6.018 \times 10^{-4} \text{ N} \cdot \text{m/rad} \quad (2.9)$$

Using equation 2.6, we obtain the following results :

$$I_{xx} = I_{yy} = 3.36 \times 10^{-3} \text{ kg} \cdot \text{m}^2 \quad (2.10)$$

$$I_{zz} = 6.71 \times 10^{-3} \text{ kg} \cdot \text{m}^2 \quad (2.11)$$

These results are physically consistent with the quad-rotor's geometry, the equality of I_{xx} and I_{yy} being due to the symmetry of the airframe, while the value of I_{zz} approximately satisfies the perpendicular axis theorem ($I_{zz} \approx I_{xx} + I_{yy}$).

These calculated constants will be used to tune the PID controller gains, ensuring that the flight controller can accurately translate pilot commands into stable motor responses. However, for the control laws to effectively use these identified physical constants, the system requires an accurate perception of its own state. Therefore, the addition of a proper sensor interface is necessary.

$$\hat{\theta}_{accel} = \arcsin\left(\frac{a_{m_{\gamma}x}}{g}\right) \quad (2.16)$$

$$\hat{\phi}_{accel} = \arctan \left(\frac{a_{m,y}}{a_{m,z}} \right) \quad (2.17)$$

It is important to note that since we assumed here that $F = 0$ and the only force acting on the sensor is gravity, the estimated pitch and roll will be very noisy, especially around vibration. To obtain more accurate results, it is common practice to use the gyroscope and combine the results of both sensors together [7][20][4].

The Gyroscope

The gyroscope is a device that measures angular velocity, and can therefore be used to determine pitch and roll.

When it rotates, a minuscule mass shifts inside it, returning a signal that is converted to a measurable electric signal. Unlike the accelerometer, it is unaffected by linear acceleration, making its estimated results immune to the external forces that make accelerometers inaccurate.

However, it still has its own flaws; it produces angle estimates that are reliable in the short-term, but that tend to drift in the long-term. The temperature of the sensor tends to affect the bias drift. To minimize the source of error, most gyros have a built-in temperature sensor to allow for the correction of temperature dependent changes [4].

The Magnetometer

The magnetometer is a device that measures the magnetic field direction in space and is used in this case to estimate the yaw angle. The particular sensor (HMC5883L) used in our case uses a magnetoresistive sensor arranged in a bridge circuit made out of nickel-iron material (Ni-Fe). Its electrical resistance varies when the applied magnetic field changes, and the resultant voltage measured across the bridge is used to get the magnetic field direction in space.

The magnetometer results are dependent on axis alignment and temperature, and the local magnetic field can be distorted by magnetic and ferrous metal objects such as the motors or electromagnetic waves from communication lines. To properly estimate the yaw angle, it is necessary to calibrate the sensor before use [10][4].

Calibration is just as necessary for the MPU6050 module. The best results for pitch and roll are obtained by combining results from both the accelerometer and the gyroscope to take advantage of their relative strengths. One way to combine the estimated angles is the use of Kalman Filters.

2.4.2 Sensor Calibration

Sensor calibration is necessary in order to read accurate results, and provide the flight controller with an accurate estimation of the quad-rotor's attitude and dynamic state.

Traditionally, specially designed mechanical platforms, such as robotic manipulators, are used for the calibration process, moving the sensor module within known rotational velocities in a set of precisely controlled orientations. These platforms being unavailable and expensive, we opted for a calibration method that doesn't require any external equipment, all while being robust and backed by research.

In order to apply this method, two primary types of errors are considered. The first type is stochastic errors such as white noise. While these errors can not be fixed using calibration, they can be modeled statistically using variance to tune the measurement noise covariance of a Kalman Filter, the implementation of which will be discussed further.

The second type is deterministic or systematic errors described as bias or offset and scale factor errors. These do not change rapidly and can be fixed using calibration [21].

Accelerometer and Gyroscope Calibration (MPU6050)

For the accelerometer calibration, the mathematical measurement model for a single axis of an accelerometer is defined as :

$$a_{raw} = S_a \cdot a_{true} + b_a + v_a \quad (2.18)$$

Where:

- a_{raw} is the raw reading from the sensor.
- a_{true} is the true physical acceleration (in units of g).
- S_a is the Scale Factor, it describes how many raw units equal $1g$.
- b_a is the Bias or Offset, it represents the raw reading when true acceleration is at 0.
- v_a is the measurement noise.

The sensor is held still in 6 orthogonal positions to calibrate all three axes independently. For each given axis, placing it facing straight up subjects it to exactly $+1g$, while placing it straight down subjects it to $-1g$. A number of 500 samples are taken to calculate the average acceleration in each specific position and determine the minimum and maximum values A_{min} and A_{max} by eliminating the measurement noise v_a .

$$A_{max} = S_a \cdot (+1g) + b_a \quad (2.19)$$

$$A_{min} = S_a \cdot (-1g) + b_a \quad (2.20)$$

By adding and subtracting the two previous equations, we can isolate the bias b_a and scale factor S_a :

$$b_a = \frac{A_{max} + A_{min}}{2} \quad (2.21)$$

$$S_a = \frac{A_{max} - A_{min}}{2} \quad (2.22)$$

Once the bias and scale factor of the accelerometer are determined, we apply a similar process to calibrate the gyroscope.

The mathematical measurement model for a single axis of an gyroscope is defined as :

$$\omega_{raw} = S_g \cdot \omega_{true} + b_g + v_g \quad (2.23)$$

Where:

- ω_{raw} is the raw digital reading from the sensor.
- ω_{true} is the true physical angular velocity (in degrees per second, $^\circ/s$).
- S_g is the Scale Factor, describing how many raw units equal $1^\circ/s$.
- b_g is the static Bias or drift, representing the raw reading when the sensor is perfectly still ($\omega_{true} = 0$).
- v_g is the high-frequency measurement noise.

To determine the static bias b_g , the sensor is placed still on a flat surface, a position in which the true angular velocity ω_{true} is 0. The measurement model simplifies to:

$$\omega_{raw} = b_g + v_g \quad (2.24)$$

A number of 2000 samples are taken in this exact state. If we consider the measurement noise v_g as zero mean Gaussian white noise, we can eliminate the noise by calculating the arithmetic mean of these samples, and therefore isolate the bias b_g for each axis.

Using this dataset of 2000 samples, we calculate the variance that will later be required to configure the Kalman Filter using the following equation :

$$\sigma_g^2 = \left(\frac{1}{N} \sum_{i=1}^N \omega_{raw,i}^2 \right) - b_g^2 \quad (2.25)$$

Where:

- σ_g^2 is the variance of the gyroscope measurement noise.
- N is the total number of recorded samples.
- $\omega_{raw,i}$ is the i -th raw sample captured during the stationary phase.

Once the bias and scale factors of the accelerometer are determined, they are stored as constants within the main flight controller software.

In order to use the sensors data in the attitude estimation, the constants are applied to the raw measurement according to the following equations :

$$a_{true} = \frac{a_{raw} - b_a}{S_a} \quad (2.26)$$

$$\omega_{true} = \frac{\omega_{raw} - b_g}{131.0} \quad (2.27)$$

The constant used in equation (2.27) represents the manufacturer's ideal theoretical scale factor for the gyroscope, which corresponds to the MPU6050 sensor's default $\pm 250^\circ/s$ sensitivity range mapped across its 16-bit resolution [21].

Magnetometer Calibration (HMC5883L)

The calibration of the magnetometer follows a similar mathematical model to the accelerometer and gyroscope. However, it addresses Soft-Iron distortions caused by ferromagnetic materials near the sensor and Hard-Iron distortions caused by permanent magnetic fields generated by the quad-rotor's motors and electronic components.

The measurement model for a single magnetometer axis is defined as:

$$m_{raw} = S_m \cdot m_{true} + b_m + v_m \quad (2.28)$$

Where:

- m_{raw} is the raw magnetic reading from the HMC5883L sensor.
- m_{true} is the true component of the Earth's magnetic field.
- S_m is the Soft-Iron scale factor.
- b_m is the Hard-Iron bias.
- v_m is the measurement noise.

The magnetometer calibration requires dynamic movement. It is continuously rotated in all three dimensions tracing a three dimensional sphere in the air, in order to capture a continuous dataset and determine the minimal and maximal values for each of the three axes.

The Hard-Iron bias b_m shifts the center of the measured magnetic 3D sphere from the origin, so calculating b_m comes down to finding the midpoint between the maximal and minimal values previously captured, as described in the following equation :

$$b_m = \frac{M_{max} + M_{min}}{2} \quad (2.29)$$

The Soft-Iron scale factor S_m on the other hand, transforms the sphere into an ellipsoid. In order to transform it back into a perfect sphere, it is determined for each axis using the following equation :

$$S_m = \frac{M_{max} - M_{min}}{2} \quad (2.30)$$

Once the two constants are determined and stored in the flight controller software, it continuously corrects the raw magnetometer measurements by subtracting the Hard-Iron bias and dividing by the Soft-Iron scale factor [21].

2.4.3 Data Acquisition Implementation

In order to determine the quad-rotor's attitude, the flight controller reads values from the MPU6050 sensor module via I2C.

I2C (Inter-Integrated Circuit protocol) is a serial communication protocol. It uses two wires, a Serial Data line (SDA) and a Serial Clock line (SCL). One of its main advantages is the possibility of communication with different devices using the same two lines, through a unique I2C address.

I2C is a bidirectional, half-duplex communication, meaning one single controller or device can send data on the bus at a time. It is also a multi-master and multi-slave protocol.

The I2C Master-Slave Pass-Through architecture (figure 2.10) is being used in this project. The master, which is the flight controller (Zybo), is the device which initiates transactions and generates a clock signal. The MPU6050 sensor module is the Auxiliary Master, acting as a bridge between the Zybo and the HMC5883L, which is the slave. The MPU6050 stores the magnetometer data in its own internal registers, so the Zybo only has to perform a single I2C read from it to get the data from all 9 axes at once. In order to apply this configuration, the HMC5883L's SCL and SDA pins are connected to the MPU6050's pins ECL and EDA, respectively (auxiliary I2C bus).

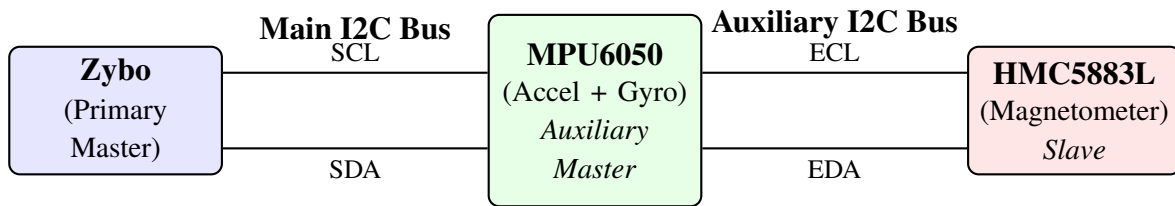


Figure 2.10: I2C Master-Slave Pass-Through architecture

This architecture presents a number of advantages:

- The data packet from the 9 axes is synchronized in time by the MPU6050.
- The bus traffic on the main I2C lines is reduced.
- The Zybo doesn't have to read from multiple sensors at once.

Once the sensors are calibrated and the data properly acquired, the Kalman Filter can be implemented in order to fuse the accelerometer and gyroscope data, and in order to determine the pitch and roll angles [12].

2.5 Data Fusion and Control Design

2.5.1 Kalman Filter Implementation for State Estimation

The theoretical foundation for the sensor fusion algorithm is derived from R. E. Kalman's seminal 1960 paper, *"A New Approach to Linear Filtering and Prediction Problems"* [11]. Kalman introduced a state-transition method for dynamic systems, allowing for the optimal estimation of signals from noisy measurements using a recursive discrete-time difference equation. In the context of this quadcopter project, the continuous physical dynamics of the drone are discretized at a 100 Hz sample rate ($\Delta t = 0.01$ s) to execute strictly within the timing constraints of the ARM Cortex-A9 processor.

Based on Kalman's state-space formulation, the system is modeled with a 2×1 state vector x_k tracking the true inclination angle θ and the gyroscope bias b :

$$x_k = \begin{bmatrix} \theta_k \\ b_k \end{bmatrix} \quad (2.31)$$

The state transition matrix F , control-input matrix B , and measurement matrix H are defined for the discrete-time system as:

$$F = \begin{bmatrix} 1 & -\Delta t \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} \Delta t \\ 0 \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 \end{bmatrix} \quad (2.32)$$

In this architecture, the gyroscope reading $\dot{\theta}_{gyro}$ acts as the control input u_k , and the accelerometer-derived angle θ_{acc} acts as the measurement z_k . The algorithm executes Kalman's recursive two-phase filter as follows:

1. Prediction (Time Update)

The state is projected forward by integrating the unbiased gyroscope rate over Δt . Translating the general prediction equation $x_{k|k-1} = Fx_{k-1|k-1} + Bu_k$ into our specific system yields:

$$\begin{bmatrix} \theta_{k|k-1} \\ b_{k|k-1} \end{bmatrix} = \begin{bmatrix} \theta_{k-1|k-1} + \Delta t(\dot{\theta}_{gyro} - b_{k-1|k-1}) \\ b_{k-1|k-1} \end{bmatrix} \quad (2.33)$$

Simultaneously, the 2×2 error covariance matrix P is extrapolated. Using the process noise covariance matrix Q (parameterized by `Q_angle` and `Q_gyroBias`), the equation $P_{k|k-1} = FP_{k-1|k-1}F^T + Q$ algebraically expands to:

$$P_{k|k-1} = \begin{bmatrix} P_{00} - \Delta t(P_{01} + P_{10} - \Delta t P_{11}) + Q_\theta & P_{01} - \Delta t P_{11} \\ P_{10} - \Delta t P_{11} & P_{11} + Q_b \end{bmatrix} \quad (2.34)$$

This unrolled matrix multiplication corresponds exactly to the FPU-optimized arithmetic lines in the `getKalman()` C function, ensuring deterministic $O(1)$ execution time.

2. Correction (Measurement Update)

Once the accelerometer measurement z_k is read via the AXI IIC bus, the innovation (pre-fit residual) y_k and the innovation covariance S_k are calculated. Here, R represents the measurement noise covariance (e.g., `R_measure_Roll = 944.52`):

$$y_k = z_k - Hx_{k|k-1} = \theta_{acc} - \theta_{k|k-1} \quad (2.35)$$

$$S_k = HP_{k|k-1}H^T + R = P_{00} + R \quad (2.36)$$

The optimal Kalman Gain vector K_k , which minimizes the a posteriori error covariance, simplifies significantly due to the 1×2 shape of H :

$$K_k = P_{k|k-1}H^T S_k^{-1} = \begin{bmatrix} P_{00}/S_k \\ P_{10}/S_k \end{bmatrix} \quad (2.37)$$

Finally, the state vector and the covariance matrix are updated (corrected) by the Kalman Gain:

$$x_{k|k} = x_{k|k-1} + K_k y_k \implies \begin{bmatrix} \theta_{k|k} \\ b_{k|k} \end{bmatrix} = \begin{bmatrix} \theta_{k|k-1} + K_0 y_k \\ b_{k|k-1} + K_1 y_k \end{bmatrix} \quad (2.38)$$

$$P_{k|k} = (I - K_k H)P_{k|k-1} = \begin{bmatrix} P_{00} - K_0 P_{00} & P_{01} - K_0 P_{01} \\ P_{10} - K_1 P_{00} & P_{11} - K_1 P_{01} \end{bmatrix} \quad (2.39)$$

By explicitly resolving these matrix transformations into scalar C operations, the computational overhead on the Cortex-A9 is minimized. The optimally estimated roll and pitch angles are then formatted into a CSV string and transmitted asynchronously via the PL-based AXI UART to the ESP-01S Wi-Fi module. This telemetry stream is ingested by a host machine running MATLAB, allowing the designer to dynamically validate the theory and fine-tune the Q and R covariance matrices against real-world propeller resonance without recompiling the embedded binary.

Complete Embedded C Implementation

The full software pipeline for the sensor fusion system is provided in Listing ???. This implementation handles the initialization of the AXI IIC interface, continuous data acquisition from the MPU6500, the application of empirically derived calibration offsets, the execution of the discrete-time Kalman filter, and the serialization of the telemetry stream for MATLAB analysis. The result is shown on the figure

2.5.2 PID Controller Tuning and Coefficient Determination

Proportional-Integral-Derivative (PID) control is a commonly used control algorithm that receives input data from sensors, calculates the difference between the actual and desired output, and adjusts the output to control a set of variables. It operates within a closed loop, through three mechanisms :

- **Proportional control**, which reacts to current error.
- **Integral control**, which addresses accumulated past errors.
- **Derivative control**, which predicts future errors.

The PID controller then sums those three components to determine the output $u(t)$ according the following equation (2.40):

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d}{dt} e(t) \quad (2.40)$$

Where $e(t)$ represents the error, and K_p , K_i and K_d the gain coefficients related to the proportional, integral and derivative terms respectively.

In order to evaluate the PID control system, a set of performance requirements is defined, as shown in the figure (2.11):

- **Rise Time** is the amount of time for the system to go from 10% to 90% of the final value.

- **Percent Overshoot**, expressed as a percentage of the final value, is the amount that the variable overshoots this final value.
- **Settling time** is the time required for the variable to settle within a certain percentage, usually 5%, of the final value.
- **Steady-State Error** is the difference between the variable and set point.

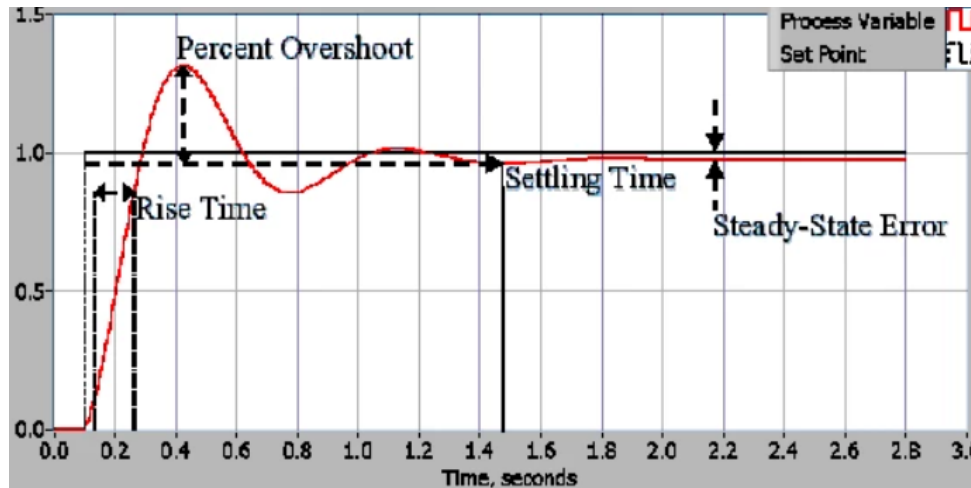


Figure 2.11: Response of a typical PID closed loop system to a step function as set point[17]

Tuning is the process of setting the optimal P, I and D gains to get an ideal response from the control system, which necessitates a clear understanding of the effect of each gain on the performance requirements[17].

The proportional component (P) determines the intensity with which the flight controller works to correct errors. Increasing K_p will generally:

- Decrease Rise Time, making the system respond faster.
- Decrease the Steady-State Error without eliminating it.
- Increase the Overshoot, which might cause oscillations and a less stable system.

The Integral component (K_i) determines the flight controller's effort to maintain the drone's attitude against external forces, such as wind or an off-centered center of gravity, to prevent the quad-rotor from drifting. Increasing K_i will generally:

- Eliminate Steady-State Error so that the system can eventually reach the set point.
- Decrease the Rise Time.
- Increase the Overshoot and Settling Time.

The Derivative component (K_d) predicts the system's behavior to prevent over correction. Increasing K_d will generally:

- Decrease the Overshoot, providing a counter-force as the variable approaches the set point.
- Decrease the Settling Time.
- Amplify vibrations and noise in the quadcopter.

Tuning can be achieved by finding a good balance between P and D gains, determining the best ratio between P and I gains, and then maximizing all three gains. This process is typically conducted

through two primary approaches. First, software environments like MATLAB/SIMULINK allow for safe **simulation-based tuning**, and help identify a stable baseline without any potential hardware damage. Then, once a stable baseline is established, the **real-life tuning** is performed on the physical quad-rotor in order to take into consideration real world factors such as sensor noise, vibrations and aerodynamic effects[17][14].

2.6 Testing and Performance Analysis

2.6.1 Hardware-in-the-Loop (HIL) Simulation and Validation

To bridge the gap between theoretical modeling and physical deployment, a Hardware-in-the-Loop (HIL) simulation was developed. This architecture integrates the physical flight controller—running on the Zybo Zynq-7010 SoC—with a high-fidelity numerical plant model in MATLAB/Simulink. This approach validates the control algorithms in real-time under dynamic constraints before actual flight testing.

System Architecture and Data Flow

The HIL setup operates as a continuous, bidirectional feedback loop. The Zybo board, equipped with an Arduino and nRF24L01 transceiver setup for robust communication, executes the time-critical PID control logic. It calculates PWM signals and transmits them to the workstation via a UART serial interface (COM4 at 115200 baud). Simulink receives these actuation commands, computes the resulting 6-Degree-of-Freedom (6-DOF) rigid body dynamics, and returns simulated IMU data (attitude and altitude) back to the hardware, completing the loop.

Plant Modeling and Experimental Parameters

The accuracy of the HIL simulation relies entirely on the fidelity of the plant model. The core of the dynamic simulation is the **6-DOF (Euler Angles)** block, which integrates the equations of motion in the body axes. To ensure the model accurately reflects the physical quadcopter, the block is parameterized using values derived from experimental measurements.

The system has a measured total mass of $m = 1.2$ kg. Furthermore, the inertia tensor, J , was experimentally determined to capture the specific mass distribution of the airframe:

$$J = \begin{bmatrix} 3.36 \times 10^{-3} & 0 & 0 \\ 0 & 3.36 \times 10^{-3} & 0 \\ 0 & 0 & 6.71 \times 10^{-3} \end{bmatrix} \text{ kg} \cdot \text{m}^2 \quad (2.41)$$

By utilizing this specific inertia matrix, the simulation accurately replicates the rotational sluggishness and agility of the real hardware, providing a reliable foundation for controller tuning.

Figure 2.12: Parameterization of the 6-DOF block highlighting the experimentally derived mass and inertia matrix.

Detailed Analysis of Simulink Blocks

The Simulink environment is structured into several functional subsystems to process data, simulate physics, and visualize output as shown in the figures (2.12),(2.14):

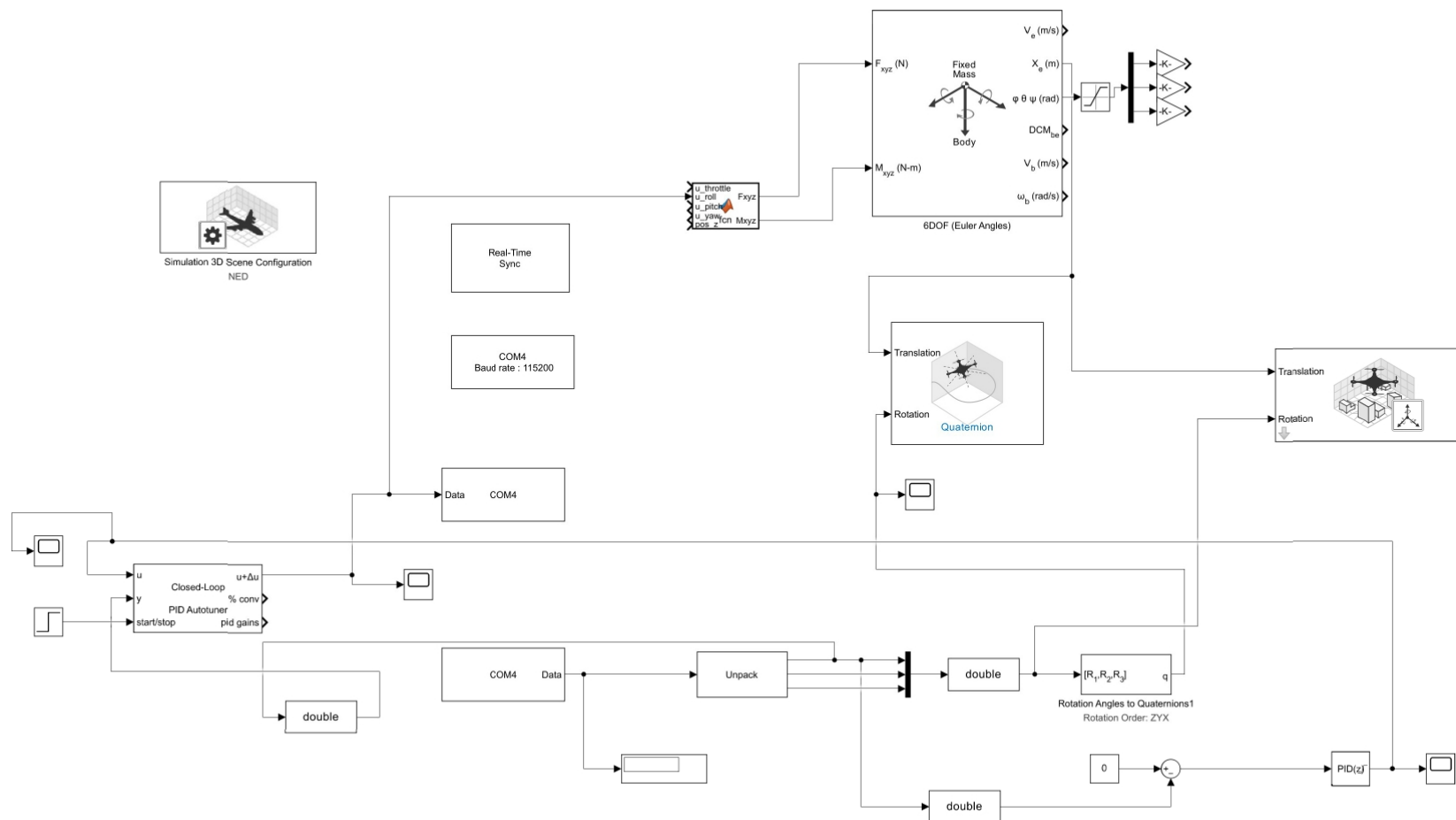


Figure 2.13: Comprehensive Simulink HIL Block Diagram. The architecture includes the UART serial communication interface for Zybo integration, the 6-DOF rigid body dynamics plant model, and the real-time synchronization kernel.

- **Serial Unpack & Data Formatting:** The Unpack block parses the incoming byte stream from the COM port, separating it into distinct throttle, roll, pitch, and yaw commands ($u_{throttle}, u_{roll}, u_{pitch}, u_{yaw}$). Subsequent double conversion blocks ensure high-precision floating-point arithmetic is maintained throughout the simulation.
- **6-DOF Equations of Motion:** This block calculates the resulting forces (F_{xyz}) and moments (M_{xyz}) applied to the rigid body, outputting linear velocities, angular rates, and Euler angles.
- **Attitude Representation:** To avoid mathematical singularities such as gimbal lock, blocks like Rotation Angles to Quaternions are utilized to represent the drone's 3D orientation internally.
- **Real-Time Synchronization:** A critical component in HIL, the Real-Time Sync block forces the Simulink solver execution to match the hardware's clock. This ensures the embedded controller receives feedback at a realistic sampling rate, preventing artificially accelerated or delayed dynamics.
- **3D Scene Configuration (NED):** Operating in the North-East-Down (NED) coordinate system, this block renders a visual representation of the drone. It allows for qualitative validation of the flight dynamics, making phenomena like unwanted oscillations immediately visible.



Figure 2.14: 3D visualization of the simulated quadcopter orientation in the NED coordinate frame.

PID Autotuning Mechanism

Tuning the closed-loop controller relies heavily on the **Closed-Loop PID Autotuner** block. Because the plant model incorporates the physical inertia matrix (2.41), the autotuner can generate precise gains tailored specifically to the Zybo-controlled hardware.

The autotuning process functions by injecting a localized test signal—typically a relay feedback or step perturbation—into the control loop during operation. The block then monitors the plant's output response to this excitation. Based on the amplitude and frequency of the resulting oscillations, the autotuner identifies a linear approximation of the plant dynamics around the current operating point.

Once the plant is estimated, the algorithm calculates the optimal Proportional, Integral, and Derivative gains based on predefined performance goals, such as desired response time and phase margin (robustness against instability). In this HIL context, the calculated discrete-time gains ($PID(z)$) are directly applicable to the Zynq-7010's digital control loop, minimizing manual trial-and-error during the transition to physical flight.

2.6.2 Closing the Loop: Flight Results

With the HIL simulation having validated the PID gains and confirmed the stability of the control architecture under simulated dynamics, the complete flight controller was deployed onto the physical Zybo platform for real-world validation. The transition from simulation to physical flight required the implementation of a robust Flight State Machine (FSM) to govern the operational lifecycle of the drone safely and predictably.

Flight State Machine Architecture

A critical requirement for any flight controller intended for physical deployment is a well-defined state management system. Rather than allowing the motors to be driven directly by the PID output at any time, the flight controller enforces a strict finite state machine that gates motor actuation behind a sequence of safety conditions. The FSM comprises four distinct states, as illustrated in Figure 2.15:

- **DISARMED:** The default power-on state. All motor outputs are held at zero regardless of any incoming commands or sensor readings. The system performs sensor initialization, IMU calibration, and Wi-Fi link establishment in this state. No actuation is possible, making it the only state in which it is safe to handle the vehicle.
- **IDLE:** The system has been armed and all pre-flight checks have passed, but the throttle command remains below the minimum flight threshold. Motors are driven at their minimum keep-alive pulse ($1000\ \mu s$) to confirm ESC responsiveness without generating significant thrust. The Kalman filter and PID integrators are active and accumulating state, ensuring a smooth transition to flight.
- **ARMED:** A transient confirmation state entered when the pilot sends an explicit arming command from the ground station. Pre-flight checks are executed — verifying IMU data validity, Wi-Fi link health, and battery voltage — before the system is permitted to advance to the IDLE or RUNNING states. If any check fails, the FSM returns to DISARMED.
- **RUNNING:** The fully operational flight state. The PID control loops are closed, and computed motor commands are dispatched to the four AXI Timer PWM channels at the full 100 Hz control rate. The FSM remains in this state as long as the throttle is above the minimum threshold and the Wi-Fi link watchdog does not time out. A link loss or an explicit disarm command from the ground station triggers an immediate transition back to IDLE, with motors ramping down gracefully rather than cutting abruptly.

Control Loop Execution

Once the FSM enters the RUNNING state, the ARM Cortex-A9 executes the closed-loop control pipeline at a fixed 100 Hz rate enforced by a hardware timer interrupt. At each tick, the Kalman filter produces fresh roll and pitch estimates from the MPU6050 data, which are fed as feedback signals into the three nested PID controllers governing roll, pitch, and yaw respectively. The PID outputs are then mixed into four individual motor throttle commands through the control allocation matrix derived from Equation (1.15), and dispatched to the AXI Timer peripherals via the memory-mapped AXI4-Lite bus.

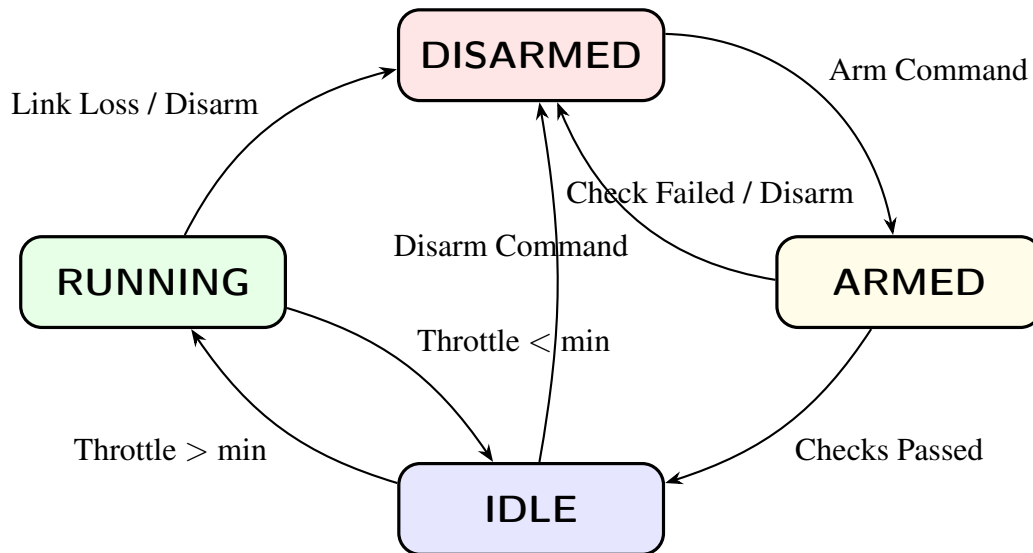


Figure 2.15: Complete State Machine for UAV Flight Controller

The gains determined through the HIL autotuning procedure served as the initial deployment values. Fine adjustments were subsequently made during tethered hover tests, where the drone was constrained by a safety line while the derivative gain K_d was trimmed to suppress the high-frequency oscillations induced by motor vibration coupling into the IMU readings.

Flight State Machine: Pseudocode

Algorithm 1 formalises the FSM logic as implemented on the ARM Cortex-A9 processor. The main loop runs continuously, with the 100 Hz control tick handled by interrupt.

Algorithm 1 Flight Controller State Machine

```

1: Initialise: state  $\leftarrow$  DISARMED
2: Calibrate IMU, establish Wi-Fi link, zero PID integrators
3:
4: loop forever ▷ Main supervisory loop
5:   cmd  $\leftarrow$  ReadWiFiCommand()
6:   imu  $\leftarrow$  ReadKalmanAngles()
7:
8:   switch state
9:     case DISARMED:
10:      SetAllMotors(0)
11:      if cmd = ARM then
12:        state  $\leftarrow$  ARMED
13:      end if
14:     case ARMED:
15:       if IMU_valid() and WiFi_linked() and Battery_OK() then
16:         state  $\leftarrow$  IDLE
17:       else
18:         state  $\leftarrow$  DISARMED ▷ Pre-flight check failed
19:       end if
20:     case IDLE:
21:       SetAllMotors(MIN_THROTTLE) ▷ Keep-alive pulse only
22:       if cmd = DISARM then
23:         state  $\leftarrow$  DISARMED
24:       else if cmd.throttle > THROTTLE_MIN then
25:         ZeroPIDIntegrators()
26:         state  $\leftarrow$  RUNNING
27:       end if
28:     case RUNNING:
29:       ▷ 100 Hz interrupt-driven control tick:
30:        $\hat{\phi}, \hat{\theta} \leftarrow \text{KalmanUpdate}(\text{imu})$ 
31:        $e_{roll} \leftarrow \text{cmd}.\phi_{ref} - \hat{\phi}$ 
32:        $e_{pitch} \leftarrow \text{cmd}.\theta_{ref} - \hat{\theta}$ 
33:        $e_{yaw} \leftarrow \text{cmd}.\psi_{ref} - \hat{\psi}$ 
34:        $\delta_{ail} \leftarrow \text{PID\_Roll}(e_{roll})$ 
35:        $\delta_{ele} \leftarrow \text{PID\_Pitch}(e_{pitch})$ 
36:        $\delta_{rud} \leftarrow \text{PID\_Yaw}(e_{yaw})$ 
37:        $[M_1, M_2, M_3, M_4] \leftarrow \text{MixerMatrix}(\text{cmd.throttle}, \delta_{ail}, \delta_{ele}, \delta_{rud})$ 
38:       UpdateMotors( $M_1, M_2, M_3, M_4$ )
39:
40:       if cmd = DISARM or not WiFi_linked() then
41:         GracefulMotorRampDown()
42:         state  $\leftarrow$  DISARMED
43:       else if cmd.throttle < THROTTLE_MIN then
44:         state  $\leftarrow$  IDLE
45:       end if
46:   end switch
47: end loop

```

2.7 Conclusion

This chapter has detailed the physical realization and experimental validation of the quad-rotor flight control system. By adopting a structured development workflow—progressing from hardware abstraction to high-level algorithmic control—the project successfully bridged the gap between theoretical modeling and physical deployment.

The following milestones summarize the implementation results:

- **Deterministic Actuation:** The integration of **AXI-based PWM generation** in the Programmable Logic (PL) ensured motor control with high precision, and the synchronization of the **SimonK ESCs** established the required linear mapping between commands and physical thrust.
- **Optimal State Estimation:** A discrete time **Kalman Filter** was implemented to fuse together the gyroscope rates and accelerometer orientation. This architecture successfully mitigated sensor noise and integration drift in order to provide a stable 100 Hz attitude estimate.
- **Validated Control Laws:** Using experimentally derived **moments of inertia** (I_{xx}, I_{yy}, I_{zz}), a **Hardware-in-the-Loop (HIL)** simulation environment allowed for precise tuning of PID gains. This minimized the risks from manual trial and error during the transition to physical flight.
- **Operational Logic:** The implementation of a four state **Finite State Machine (FSM)** that comprises Disarmed, Armed, Idle, and Running states ensures a predictable operational lifecycle.

The convergence of these subsystems confirms that the SoC-based flight controller is capable of maintaining stable flight dynamics. With the core stabilization layer now fully validated, the system serves as a robust platform for future autonomous navigation and higher-level mission tasks.

Conclusion and Future Work

This project has demonstrated the successful design and implementation of a high-performance, deterministic flight controller by fully exploiting the heterogeneous architecture of the Zynq-7010 System-on-Chip. By adopting a rigorous hardware/software co-design methodology, time-critical operations such as PWM signal generation and sensor interfacing were offloaded to the Programmable Logic (PL), ensuring strict temporal determinism and eliminating jitter. In parallel, the Processing System (PS), built around the ARM Cortex-A9, was dedicated to computationally intensive tasks, notably the execution of a discrete-time Kalman filter at 100 Hz and the implementation of multi-loop PID control laws. This architectural partitioning proved to be a decisive factor in achieving a responsive and stable control system, effectively bridging the gap between noisy raw sensor measurements and precise motor actuation.

After the embedded implementation was complete, the development of a Hardware in the Loop (HIL) simulation environment added an important contribution to this project. By integrating the real controller with a very similar dynamic model in MATLAB/Simulink, it was possible to validate the completed control architecture, including the Kalman filter, PID loop, and Finite State Machine (FSM), under realistic conditions before physical deployment. This approach accelerated the tuning process of PID gains while significantly reducing safety risks, allowing the evaluation of systematic performance without exposing the hardware platform to the risk of failure during testing.

The experimental results confirm that the system can achieve stable attitude control and is robust against disturbances, which therefore validates both the theoretical modeling and implementation. More broadly, this work highlights how relevant SoC based architectures are in embedded control systems, in particular where the applications presents real time constraints, computational complexity, and system reliability that must be satisfied simultaneously .

Looking forward, there naturally are several avenues for further development from this foundation. The next step involves the transition from simulation based validation to controlled physical testing using a 3-axis gimbal rig. This setup will allow the quadrotor to rotate freely in roll, pitch, and yaw, all while being mechanically constrained, providing a safe and controllable platform for refining control performance. In particular, it will allow for more precise tuning of the components of the PID controller and facilitate the analysis of high frequency mechanical vibrations that are difficult to capture in simulation but critically affect real world stability.

In addition, computational and logical resources remain available on the Zynq platform, which opens the door to more advanced functionalities. Future work will focus on providing the system with full autonomy by integrating higher-level navigation capabilities, such as GPS-based waypoint tracking, barometric altitude regulation, and sensor fusion for position estimation. Furthermore, the programmable logic offers the possibility to integrate accelerated compute intensive tasks such as real-time computer vision, therefore enabling onboard obstacle detection and avoidance without having to compromise the control loop performance.

Finally, this project establishes an extensible foundation for the development of next generation UAV control systems. By combining both deterministic hardware execution and flexible software intelligence, the proposed architecture not only meets the requirements of flight stabilization but also offers a pathway toward intelligent, autonomous aerial platforms that are capable of operating in complex environments.

Bibliography

- [1] Ai-Thinker Technology. (2019). *ESP-01S WiFi Module Datasheet*. Available at: <https://www.ai-thinker.com/> (Accessed: April 2026).
- [2] ArduPilot Dev Team. (2024). *Electronic Speed Controller (ESC) Calibration*. Available at: <https://ardupilot.org/copter/docs/esc-calibration.html> (Accessed: April 2026).
- [3] Bouabdallah, S., Noth, A., & Siegwart, R. (2004). *PID vs LQ Control Techniques Applied to an Indoor Micro Quadrotor*. IEEE International Conference on Intelligent Robots and Systems (IROS).
- [4] CH Robotics. (2022). *Fundamentals of Attitude Estimation: Sensors for Estimation*. Available at: <https://www.chrobotics.com/library/sensors-for-estimation> (Accessed: April 2026).
- [5] Digilent, Inc. (2018). *Zybo Z7 Board Reference Manual*. Available at: <https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual> (Accessed: November 2025).
- [6] Espressif Systems IoT Team. (2016). *ESP8266 AT Instruction Set (Version 1.5.4)*. Espressif Systems.
- [7] Fisher, C. J. (2010). *AN-1057: Using an Accelerometer for Inclination Sensing*. Analog Devices, Inc.
- [8] Generic Manufacturer. *30A BLDC ESC Product Manual: Brushless Electronic Speed Controller User Guide*. [Technical Datasheet].
- [9] Haydon Kerk Pittman. (2023). *What is the Difference Between a Brush and a Brushless Motor?*. Available at: <https://www.haydonkerkpittman.com/learningzone/blog/2023/august/what-is-the-difference-between-a-brush-and-a-brushless-motor> (Accessed: April 2026).
- [10] Honeywell. (2013). *HMC5883L 3-Axis Digital Compass IC Datasheet*. Available at: <https://www.gotronic.fr/pj-956.pdf> (Accessed: April 2026).
- [11] Kalman, R. E. (1960). *A New Approach to Linear Filtering and Prediction Problems*. Journal of Basic Engineering, Research Institute for Advanced Study.
- [12] Kumar, P. V., et al. (2018). *A Review Paper on I2C Communication Protocol*. International Journal of Advance Research, Ideas and Innovations in Technology (IJARIIT).
- [13] Liang, O. (2023). *ESC Guide for FPV Drones*. Available at: <https://oscarliang.com/esc/> (Accessed: April 2026).
- [14] Liang, O. (2023). *PID Tuning Guide for Quadcopters: Everything You Need to Know*. Available at: <https://oscarliang.com/pid/> (Accessed: April 2026).

- [15] McRuer, D., Ashkenas, I., & Graham, D. (1973). *Aircraft Dynamics and Automatic Control*. Technical Report No. 129-1, Systems Technology, Inc., Hawthorne, California.
- [16] Mead, J., Bobda, C., & Whitaker, T. J. (2016). *Defeating drone jamming with hardware sand-boxing*. 2016 IEEE Asian Hardware-Oriented Security and Trust (AsianHOST), Yilan, Taiwan, pp. 1-6. doi: 10.1109/AsianHOST.2016.7835557.
- [17] National Instruments. (2024). *PID Theory Explained*. Available at: <https://www.ni.com/en/shop/labview/pid-theory-explained.html> (Accessed: April 2026).
- [18] Pascual, S., & García, J. (Eds.). (2017). *Advanced UAV Aerodynamics, Flight Stability and Control: Novel Concepts, Theory and Applications* (Chapters 20 & 21). John Wiley & Sons.
- [19] Quan, Q. (2017). *Introduction to Multicopter Design and Control*. Springer Nature.
- [20] SparkFun Electronics. (2023). *Accelerometer Basics*. Available at: <https://learn.sparkfun.com/tutorials/accelerometer-basics> (Accessed: April 2026).
- [21] Tedaldi, D., Pretto, A., & Menegatti, E. (2014). *A Robust and Easy to Implement Method for IMU Calibration without External Equipments*. Proceedings of the IEEE International Conference on Robotics and Automation (ICRA).